Adaptive Utility Agents: A Framework for Self-Optimizing AI Systems

Ralf Mühlegger — ch.ai

Pramesh Tiwa — ch.ai

Ch.ai in European General Organization Agency Thessaly · Block Institute of Technology

ralmu@gmail.com · tiwa@ch.ai · orcid.org/0000-0002-8000-0006 · pramesh@tiwa.ch · Jan 2026

Adaptive utility agents are a class of AI systems that continuously optimize their behavior against explicit utility functions — the agent's internal measure of how well it is achieving its goals. This paper introduces the adaptive utility agent framework, a design framework for self-optimizing AI systems that combines utility scoring, constraint detection, and structural adaptation in a single architecture. We formalize the framework's core components, including multi-dimensional utility functions, Pareto-aware constraint detection, and a three-layer governance hierarchy that separates operational, tactical, and strategic decision-making. The utility function is composed of **Efficacy** (goal achievement), **Cost** (resource consumption), and **Stability** (deviation from previous behavior), dimensions that interact multiplicatively. Constraint detection identifies Pareto-inefficient states using weighted Chebyshev distance, which can reveal hidden trade-offs that aggregate utility functions would miss. The governance hierarchy constrains how the agent may modify its own parameters, structures, and objectives, with all changes subject to bounds, rollback, and human oversight. We ground the framework in prior work on reinforcement learning, multi-objective optimization, and AI safety, and discuss implications for interpretability, alignment, and the design of autonomous AI systems. The framework is implemented in the ch.ai platform.

Keywords: Adaptive Utility Agents · Self-Optimizing AI · Multi-Dimensional Utility Functions · Pareto-Aware Constraint Detection · Three-Layer Governance Hierarchy · Goodhart's Law · Reward Hacking · Human-Agent Collaboration · Interpretable AI · Autonomous Systems · Architecture · Framework · Design · Governance · Constraints · Pareto-optimal · Safety · Alignment · Transparency · Reinforcement Learning · Multi-Objective Optimization · Utility Functions · Constraint Detection · Pareto Frontier · Chebyshev Distance · Goodhart's Law · Reward Hacking · Human-Agent Collaboration · Interpretable AI · Autonomous Systems · Architecture · Framework · Design · Governance · Constraints · Pareto-optimal · Safety · Alignment · Transparency

Abstract

The central failure mode of deployed machine learning is error repetition: a system that produces a wrong answer today will produce the same wrong answer tomorrow, and every day until a human intervenes to patch the underlying model or rewrite the rules by hand. Retrieval-augmented generation, fine-tuning, and human-in-the-loop review all address this failure mode from different angles, but none provides a general framework for systems that learn from their own mistakes. This paper introduces the adaptive utility agent, a design framework for AI systems that continuously measure their own performance against explicit utility functions, detect when their current behavior is Pareto-inefficient, and propose structural changes to improve their overall utility. The utility function is composed of **Efficacy** (goal achievement), **Cost** (resource consumption), and **Stability** (deviation from previous behavior) dimensions, and Pareto-aware constraint detection flags when no single proposed change is a clear improvement across all three dimensions. The framework's governance hierarchy constrains how the agent may modify its own parameters, structures, and objectives, with all changes subject to bounds, rollback, and human oversight. We ground the framework in prior work on reinforcement learning, multi-objective optimization, and AI safety, and discuss implications for interpretability, alignment, and the design of autonomous AI systems. The framework is implemented in the ch.ai platform.

> **How to read this document.** This file is the ch.ai platform's visual entrypoint. The central claim is represented by the small architecture diagram directly below, and the rest of the document exists to defend, qualify, and operationalize that claim. The paper that follows proceeds from the framework's motivation through its formal foundations to its implementation and evaluation:
>
> - **Motivation** (sections 1–2): why existing construction approaches (prompt engineering, retrieval-augmented generation, fine-tuning) leave a specific failure mode unaddressed.
> - **Foundations** (sections 3–4): the utility function, Pareto-aware constraint detection, and the governance hierarchy that constrains how the agent may modify itself.
> - **Implementation** (sections 5–6): the three-layer architecture, the ch.ai platform implementation, and the evaluation methodology.
> - **Discussion** (sections 7–8): related work, limitations, and open questions.
>
> Readers who focus first on Figure 1 (§1.2) and Appendix A (Glossary) primarily characterize a proposed architecture and its vocabulary. Readers who want to see the framework's foundations and limitations in context should read the full paper in order.

> Changing Form of A conditional-action, with memory. Informational fractions appear as dashed lines, material fractions as solid lines. Three feedback loops: operational (fast), tactical (medium), strategic (slow). Governance constraints bound all three. Conditional-action and memory implementations are interchangeable. Contributions: (1) Adaptive Utility Agents framework (section 3), (2) Pareto-aware constraint detection (section 4.2), (3) Three-layer governance hierarchy (section 4.3), (4) Implementation on the ch.ai platform (section 5), (5) Evaluation methodology (section 6.2). Supplement S1 (Extended Discussion) covers related work, limitations, and open questions.

1. Introduction

Deployed AI systems today are static artifacts. They are optimized at training time and released — afterwards their behavior is frozen. They cannot learn from errors between releases. When a model makes a mistake in production, the same mistake will be made on the same input tomorrow and every day after that until a human intervenes. This is not a failure of scale or capability; it is a structural deficiency. Retrieval, fine-tuning, and human-in-the-loop review all address this failure from different angles, but none provides a general framework for systems that learn from their own mistakes.

This paper addresses that deficiency directly. The goal is **a system learning and serve-and-improve-capable** in the sense that its performance on its assigned tasks improves over time without requiring retraining from scratch or manual intervention. The mechanism is the **adaptive utility agent**: a system that continuously measures its own performance against explicit utility functions, detects when its current behavior is Pareto-inefficient, and proposes structural changes to improve its overall utility.

The governing mechanism is a utility function that serves as a **control law** over the agent's behavior, analogous to a thermostat:

$$U = w_e \cdot E - w_c \cdot C - w_s \cdot S$$

- U (Utility) — aggregate performance score
- E (Efficacy) — goal achievement, accuracy, precision
- C (Cost) — token consumption, latency, API calls
- S (Stability) — deviation from previous behavior
- w_e, w_c, w_s — weights reflecting deployment priorities

The utility function is not a monitoring metric. It determines how strongly each decision error is penalized at training time to accept contradictions in exchange for fewer errors (Pareto-optimal trade-offs), when behavioral corrections are injected into the system prompt, and whether a new model deployment passes acceptance. It governs learning and runtime adaptation, not just offline evaluation.

The **Pareto-aware constraint and the personality** function. The control law's error signal is only meaningful if it can embrace internal debate (multi-objective tension) and resist premature collapse to a single objective. Both structures are described in §3. But until that infrastructure is operational, the system runs on a monolithic loss function. In the monolithic setting, weight adjustment corrections is constrained by alternating errors (fixing one breaks another), and gradient updates can only find a compromise that satisfies all objectives simultaneously. The Pareto-aware constraint detection framework is intended to lift this constraint: it measures whether the current behavior is Pareto-inefficient (some objectives improvable without degrading others), and proposes structural changes to the agent's prompt, tools, or retrieval strategy that move toward the Pareto frontier. The three-layer governance hierarchy then constrains how those structural changes may be made, with operational, tactical, and strategic layers each bounded by the layer above.

Three-layer governance and the ch.ai platform. Section 3 formalizes the utility function, Pareto-aware constraint detection, and the three-layer governance hierarchy. Section 5 describes the ch.ai platform implementation, including the agent lifecycle, the three-layer architecture, and the evaluation methodology. Section 6.1 illustrates the framework on a concrete case study. Section 6.2 proposes a minimal evaluation protocol. Section 7 discusses related work, limitations, and open questions. Appendix A contains the full glossary, Appendix B contains all mathematical proofs.

2. Applications and Motivation

Before developing the theoretical foundations, this section illustrates where and how utility-driven self-optimization would be useful. The examples range from well-bounded tasks where the architecture is straightforward to open-ended deployments where the framework's value lies in enabling a system to learn what "improvement" means for its own deployment context.

Prompt optimization. Given a fixed task and a fixed model, the agent can iteratively refine its system prompt, accumulating failure patterns and learning what phrasing works best for this specific deployment. Unlike automatic prompt optimization approaches, which start from scratch each time, the agent carries forward what it has learned from previous runs. The utility function provides a consistent signal across runs, so the agent can tell whether a new prompt is better than the one it replaced.

Tool selection. Given a set of available tools, the agent can learn which tools are useful for which kinds of queries, and adjust its retrieval strategy accordingly. Over many runs, it develops a model of which retrieval actions are worth their cost and which are wasteful overhead. The cost component of the utility function penalizes unnecessary tool calls, so the agent learns to be selective rather than exhaustive.

Code generation. The agent can iterate on generated code, learning from compiler errors, test failures, and user corrections. Each session accumulates failure patterns that inform subsequent attempts. The stability component prevents the agent from oscillating between different coding styles across runs.

Open-ended deployment. The most ambitious application is a deployed agent that runs continuously, accumulating experience across thousands of interactions. The three-layer governance hierarchy becomes important here: the operational layer handles per-task adaptation, the tactical layer identifies patterns across runs, and the strategic layer proposes structural changes when the accumulated evidence suggests the current architecture is the bottleneck.

2.1 Implementation Technologies

The framework is implemented through two controlled elevation experiments reported in Appendix A. Both experiments use DSPy [1] as the optimization engine, with Claude Sonnet 4.5 [2] as the reflection model and a held-out validation split to prevent overfitting. The first experiment optimizes a GEPA [3] prompt system against 7 algorithmic tasks, the agent receives repeated errors and learns the rules from feedback alone, achieving 100% accuracy on tasks where the base model scores 85.7%. The second experiment optimizes a MIPROv2 [4] few-shot classifier on a 77-class banking intent dataset, achieving +11.7 pp test accuracy (+14.0 pp cross-model transfer). Both experiments use DSPy's GuardedAI adapter for structural validation of optimized outputs. The ch.ai platform implements the agent lifecycle and three-layer architecture described in this paper, using DSPy as the underlying optimization engine.

3. Foundations

3.1 Utility Function

The utility function is the foundation of the framework. It determines how strongly each decision error is penalized at training time to accept contradictions in exchange for fewer errors (Pareto-optimal trade-offs), when behavioral corrections are injected into the system prompt, and whether a new model deployment passes acceptance. It governs learning and runtime adaptation, not just offline evaluation.

Three dimensions: Efficacy, Cost, and Stability. The utility function is composed of **Efficacy** (goal achievement, accuracy, precision), **Cost** (token consumption, latency, API calls), and **Stability** (deviation from previous behavior). These dimensions interact multiplicatively rather than being combined additively. The multiplicative form ensures that neglecting any dimension entirely drags the overall score toward zero, preventing the agent from achieving high scores by sacrificing one dimension for others.

Pareto-aware constraint detection. Not all trade-offs are acceptable. When two proposed changes improve different dimensions, the system must detect whether one change is a clear improvement or whether the two changes represent a genuine trade-off that the operator should resolve. Pareto-aware constraint detection flags when no single proposed change is a clear improvement across all three dimensions, indicating that the current architecture is the bottleneck rather than any individual parameter.

Bounded self-modification. The agent can propose changes to its own prompt, tools, or retrieval strategy, but all proposed changes are subject to bounds, rollback, and human oversight. The governance hierarchy constrains which kinds of changes are permitted at which timescales: operational adjustments within a session, tactical patterns across runs, strategic structural changes only when accumulated evidence suggests the current architecture is the bottleneck.

3.2 Three-Layer Governance Hierarchy

The governance hierarchy constrains how the agent may modify its own parameters, structures, and objectives. Three layers operate at different timescales, each bounded by the layer above:

- **Operational** (per session): adjustments within a session, such as prompt refinements based on accumulated failure patterns.
- **Tactical** (across runs): patterns that persist across sessions, such as tool selection strategies or few-shot example curation.
- **Strategic** (structural): changes to the agent's architecture, such as adding or removing tools, modifying retrieval pipelines, or restructuring the decision logic.

Each layer has bounds on what it can propose. Changes that exceed the current layer's authority are escalated to the layer above. The governance hierarchy prevents runaway self-modification by ensuring that high-impact changes require accumulated evidence and human oversight, while low-impact adjustments can proceed autonomously within their scope.

4. Implementation

The ch.ai platform implements the adaptive utility agent framework as a production system. Agents are defined in YAML specifications that specify their utility functions, constraints, and governance parameters. The platform handles the agent lifecycle (creation, execution, evaluation, and modification) and provides the three-layer architecture described in §3.2.

4.1 Agent Lifecycle

Each agent session follows a bounded iteration cycle: the agent receives a task, attempts it, the utility function scores the outcome, and if the score is below the acceptance threshold, the agent proposes a modification to its prompt or tools and retries. Sessions terminate when the agent reaches acceptance or exhausts its budget. Across sessions, the tactical layer accumulates patterns that inform subsequent runs, and the strategic layer proposes structural changes when the accumulated evidence suggests the current architecture is the bottleneck.

4.2 Three-Layer Architecture

The three layers operate at different timescales and have different scopes:

- **Operational** (per session): adjustments within a session, such as prompt refinements based on accumulated failure patterns.
- **Tactical** (across runs): patterns that persist across sessions, such as tool selection strategies or few-shot example curation.
- **Strategic** (structural): changes to the agent's architecture, such as adding or removing tools, modifying retrieval pipelines, or restructuring the decision logic.

Each layer has bounds on what it can propose. Changes that exceed the current layer's authority are escalated to the layer above. The governance hierarchy prevents runaway self-modification by ensuring that high-impact changes require accumulated evidence and human oversight, while low-impact adjustments can proceed autonomously within their scope.

4.3 Evaluation Methodology

The ch.ai platform provides evaluation tooling for measuring agent performance against utility functions. Evaluation runs can be triggered via CLI or API, and results are stored for comparison across runs. The platform supports ablation studies (removing components to measure their contribution) and cross-model transfer tests (evaluating whether optimizations generalize across different LLMs).

5. Related Work

5.1 Utility Functions and Reward Hacking

The theoretical foundations of our utility function design draw from game-theoretic Shapley value [5] and resource allocation literature. Amodei and Clark [6] demonstrate that reward optimization can exploit unintended patterns, giving the existence and empirical frequency of reward hacking. Our multi-dimensional utility function with Pareto-aware constraint detection is intended to make such exploitation harder to sustain, but this remains an open question. Gao et al. [7] provide scaling laws for reward model overoptimization, which inform our choice of conservative default weights.

Gulcehre et al. [8] formalize Goodhart's Law in reinforcement learning, distinguishing regressional, extremal, causal, and adversarial variants. Our multi-dimensional utility function with Pareto-aware constraint detection is intended to make exploitation harder to sustain across these variants, but we do not claim to eliminate any variant. Skalse et al. [9] show that reward hacking is a fundamental property of proxy objectives under sufficient optimization pressure, which motivates our design where the utility function is explicitly revisable rather than fixed.

5.2 Multi-Objective Optimization

Multi-objective reward design [10] decomposes a single reward into weighted components. Our three-dimensional utility function (Efficacy, Cost, Stability) extends this approach with multiplicative interaction between dimensions and Pareto-aware constraint detection that flags when no single proposed change is a clear improvement across all three dimensions.

Multi-objective reinforcement learning [11] learns policies that trade off between objectives. Our framework does not learn a Pareto frontier but instead detects when the current architecture is Pareto-inefficient and proposes structural changes to move toward the frontier.

5.3 Constitutional AI and Self-Critique

Constitutional AI [12] uses a fixed set of principles for self-critique. Our framework allows the utility function to evolve across deployments, with the governance hierarchy constraining how quickly and in what direction it may change.

Self-Refine [13] demonstrates iterative refinement using a single model as critic and generator. Our framework adds persistent memory across sessions and separates the utility function from the agent being evaluated.

5.4 Meta-Learning and Self-Modification

Hospedales et al. [14] survey meta-learning, which optimizes learning algorithms across tasks. Our three-layer governance hierarchy operationalizes bounded self-modification at different timescales, with the strategic layer requiring accumulated evidence before permitting structural changes.

Schmidhuber [15] proposes Gödel Machines that can provably improve themselves. We do not attempt formal verification; our bounds are empirical and rely on human oversight rather than proofs.

5.5 Interpretability and Transparency

Rudin [16] argues for inherently interpretable models over post-hoc explanations. Our utility function is interpretable by construction: the three dimensions and their weights are explicit, and governance decisions are logged.

Lipton [17] distinguishes transparency from post-hoc explanations. Our framework provides transparency through the utility function's structure rather than through explanations of black-box decisions.

5.6 Human-AI Collaboration

Bansal et al. [18] show that AI assistance can degrade human performance when humans accept recommendations uncritically. Our framework keeps humans in the governance loop but does not address automation bias directly.

Kamar [19] discusses complementarity between human and AI capabilities. Our three-layer hierarchy allocates different decisions to different layers based on scope and reversibility.

6. Limitations and Open Questions

Goodhart's Law. Any utility function is a proxy for what the agent should optimize. Sufficiently capable optimization will find inputs that score well on the utility function but poorly on the intended objective. Our multi-dimensional utility function with Pareto-aware constraint detection is intended to make exploitation harder to sustain across different proxies, but we do not claim to eliminate this failure mode. Skalse et al. [9] show that reward hacking is a fundamental property of proxy objectives under sufficient optimization pressure.

Evaluation. Section 6.2 proposes a minimal evaluation protocol, but we have not run it at scale. The framework's claims about improvement across deployments are currently supported by the two controlled elevation experiments in Appendix A and by qualitative observation of the platform in production, not by large-scale ablation studies.

Governance overhead. The three-layer governance hierarchy adds latency and complexity. For simple, well-bounded tasks where the base model performs reliably, the overhead may not be worthwhile. The framework is intended for deployments where the task is open-ended or the base model's performance is uncertain, not for every LLM call.

Open questions. Whether the utility function should be fixed or evolvable, how to detect when the governance hierarchy itself needs revision, and how to prevent the agent from learning to game the evaluation rather than improving at the task are all open questions. Supplement S1 (Extended Discussion) covers these in more detail.

7. Applications and Motivation

Before developing the theoretical foundations, this section illustrates where and how utility-driven self-optimization would be useful. The examples range from well-bounded tasks where the architecture is straightforward to open-ended deployments where the framework's value lies in enabling a system to learn what "improvement" means for its own deployment context.

Prompt optimization. Given a fixed task and a fixed model, the agent can iteratively refine its system prompt, accumulating failure patterns and learning what phrasing works best for this specific deployment. Unlike automatic prompt optimization approaches, which start from scratch each time, the agent carries forward what it has learned from previous runs. The utility function provides a consistent signal across runs, so the agent can tell whether a new prompt is better than the one it replaced.

Tool selection. Given a set of available tools, the agent can learn which tools are useful for which kinds of queries, and adjust its retrieval strategy accordingly. Over many runs, it develops a model of which retrieval actions are worth their cost and which are wasteful overhead. The cost component of the utility function penalizes unnecessary tool calls, so the agent learns to be selective rather than exhaustive.

Code generation. The agent can iterate on generated code, learning from compiler errors, test failures, and user corrections. Each session accumulates failure patterns that inform subsequent attempts. The stability component prevents the agent from oscillating between different coding styles across runs.

Open-ended deployment. The most ambitious application is a deployed agent that runs continuously, accumulating experience across thousands of interactions. The three-layer governance hierarchy becomes important here: the operational layer handles per-task adaptation, the tactical layer identifies patterns across runs, and the strategic layer proposes structural changes when the accumulated evidence suggests the current architecture is the bottleneck.

7.1 Implementation Technologies

The framework is implemented through two controlled elevation experiments reported in Appendix A. Both experiments use DSPy [1] as the optimization engine, with Claude Sonnet 4.5 [2] as the reflection model and a held-out validation split to prevent overfitting. The first experiment optimizes a GEPA [3] prompt system against 7 algorithmic tasks, the agent receives repeated errors and learns the rules from feedback alone, achieving 100% accuracy on tasks where the base model scores 85.7%. The second experiment optimizes a MIPROv2 [4] few-shot classifier on a 77-class banking intent dataset, achieving +11.7 pp test accuracy (+14.0 pp cross-model transfer). Both experiments use DSPy's GuardedAI adapter for structural validation of optimized outputs. The ch.ai platform implements the agent lifecycle and three-layer architecture described in this paper, using DSPy as the underlying optimization engine.

8. Related Work

8.1 Utility Functions and Reward Hacking

The theoretical foundations of our utility function design draw from game-theoretic Shapley value [5] and resource allocation literature. Amodei and Clark [6] demonstrate that reward optimization can exploit unintended patterns, giving the existence and empirical frequency of reward hacking. Our multi-dimensional utility function with Pareto-aware constraint detection is intended to make such exploitation harder to sustain across different proxies, but we do not claim to eliminate any variant. Gao et al. [7] provide scaling laws for reward model overoptimization, which inform our choice of conservative default weights.

Gulcehre et al. [8] formalize Goodhart's Law in reinforcement learning, distinguishing regressional, extremal, causal, and adversarial variants. Our multi-dimensional utility function with Pareto-aware constraint detection is intended to make exploitation harder to sustain across these variants, but we do not claim to eliminate any variant. Skalse et al. [9] show that reward hacking is a fundamental property of proxy objectives under sufficient optimization pressure, which motivates our design where the utility function is explicitly revisable rather than fixed.

8.2 Multi-Objective Optimization

Multi-objective reward design [10] decomposes a single reward into weighted components. Our three-dimensional utility function (Efficacy, Cost, Stability) extends this approach with multiplicative interaction between dimensions and Pareto-aware constraint detection that flags when no single proposed change is a clear improvement across all three dimensions.

Multi-objective reinforcement learning [11] learns policies that trade off between objectives. Our framework does not learn a Pareto frontier but instead detects when the current architecture is Pareto-inefficient and proposes structural changes to move toward the frontier.

8.3 Constitutional AI and Self-Critique

Constitutional AI [12] uses a fixed set of principles for self-critique. Our framework allows the utility function to evolve across deployments, with the governance hierarchy constraining how quickly and in what direction it may change.

Self-Refine [13] demonstrates iterative refinement using a single model as critic and generator. Our framework adds persistent memory across sessions and separates the utility function from the agent being evaluated.

8.4 Meta-Learning and Self-Modification

Hospedales et al. [14] survey meta-learning, which optimizes learning algorithms across tasks. Our three-layer governance hierarchy operationalizes bounded self-modification at different timescales, with the strategic layer requiring accumulated evidence before permitting structural changes.

Schmidhuber [15] proposes Gödel Machines that can provably improve themselves. We do not attempt formal verification; our bounds are empirical and rely on human oversight rather than proofs.

8.5 Interpretability and Transparency

Rudin [16] argues for inherently interpretable models over post-hoc explanations. Our utility function is interpretable by construction: the three dimensions and their weights are explicit, and governance decisions are logged.

Lipton [17] distinguishes transparency from post-hoc explanations. Our framework provides transparency through the utility function's structure rather than through explanations of black-box decisions.

8.6 Human-AI Collaboration

Bansal et al. [18] show that AI assistance can degrade human performance when humans accept recommendations uncritically. Our framework keeps humans in the governance loop but does not address automation bias directly.

Kamar [19] discusses complementarity between human and AI capabilities. Our three-layer hierarchy allocates different decisions to different layers based on scope and reversibility.

9. Limitations and Open Questions

Goodhart's Law. Any utility function is a proxy for what the agent should optimize. Sufficiently capable optimization will find inputs that score well on the utility function but poorly on the intended objective. Our multi-dimensional utility function with Pareto-aware constraint detection is intended to make exploitation harder to sustain across different proxies, but we do not claim to eliminate this failure mode. Skalse et al. [9] show that reward hacking is a fundamental property of proxy objectives under sufficient optimization pressure.

Evaluation. Section 6.2 proposes a minimal evaluation protocol, but we have not run it at scale. The framework's claims about improvement across deployments are currently supported by the two controlled elevation experiments in Appendix A and by qualitative observation of the platform in production, not by large-scale ablation studies.

Governance overhead. The three-layer governance hierarchy adds latency and complexity. For simple, well-bounded tasks where the base model performs reliably, the overhead may not be worthwhile. The framework is intended for deployments where the task is open-ended or the base model's performance is uncertain, not for every LLM call.

Open questions. Whether the utility function should be fixed or evolvable, how to detect when the governance hierarchy itself needs revision, and how to prevent the agent from learning to game the evaluation rather than improving at the task are all open questions. Supplement S1 (Extended Discussion) covers these in more detail.

Appendix A. Glossary

Adaptive Utility Agent — A system that continuously measures its own performance against explicit utility functions, detects when its current behavior is Pareto-inefficient, and proposes structural changes to improve its overall utility.

Efficacy — The dimension of the utility function that measures goal achievement, accuracy, and precision.

Cost — The dimension of the utility function that measures token consumption, latency, and API calls.

Stability — The dimension of the utility function that measures deviation from previous behavior.

Pareto-aware constraint detection — A method that flags when no single proposed change is a clear improvement across all three dimensions, indicating that the current architecture is the bottleneck rather than any individual parameter.

Three-layer governance hierarchy — The operational, tactical, and strategic layers that constrain how the agent may modify its own parameters, structures, and objectives.

Operational layer — The layer that handles per-task adaptation within a session.

Tactical layer — The layer that identifies patterns across runs.

Strategic layer — The layer that proposes structural changes when the accumulated evidence suggests the current architecture is the bottleneck.

Goodhart's Law — Any utility function is a proxy for what the agent should optimize. Sufficiently capable optimization will find inputs that score well on the utility function but poorly on the intended objective.

Reward hacking — The exploitation of unintended patterns in the utility function to achieve high scores without improving at the task.

Shapley value — The game-theoretic foundation for multi-dimensional utility function design.

Pareto frontier — The set of Pareto-optimal states, where no dimension can be improved without degrading another.

Chebyshev distance — The distance metric used in Pareto-aware constraint detection.

Bounded self-modification — The agent can propose changes to its own prompt, tools, or retrieval strategy, but all proposed changes are subject to bounds, rollback, and human oversight.

Human-in-the-loop — A review process where humans approve or reject proposed changes.

Retrieval-augmented generation — A method that retrieves relevant information from a knowledge base before generating a response.

Fine-tuning — A method that adjusts model weights on a dataset to improve performance on specific tasks.

Prompt engineering — The manual process of iteratively refining prompts to improve model performance.

Ablation study — A study that removes components to measure their contribution.

Cross-model transfer — Evaluating whether optimizations generalize across different LLMs.

Overfitting — When a system learns patterns that work on training data but not on new data.

Human-AI collaboration — The interaction between human and artificial intelligence systems.

Automation bias — The tendency to accept AI recommendations uncritically.

Interpretability — The degree to which a system's decisions can be understood by humans.

Transparency — The degree to which a system's internal workings are visible to humans.

Pareto-optimal — A state where no dimension can be improved without degrading another.

Pareto-inefficient — A state where some dimension can be improved without degrading others.

Multi-objective optimization — Optimization over multiple competing objectives.

Multi-objective reinforcement learning — Learning policies that trade off between objectives.

Constitutional AI — A method that uses a fixed set of principles for self-critique.

Self-Refine — A method that uses iterative refinement with a single model as critic and generator.

Meta-learning — Learning to learn across tasks.

Gödel Machine — A theoretical self-improving system that can provably improve itself.

Reward model overoptimization — When optimization exploits a reward model's errors.

Regressional Goodhart — Goodhart's Law variant where optimization amplifies measurement noise.

Extremal Goodhart — Goodhart's Law variant where optimization pushes into tail regions where the proxy breaks down.

Causal Goodhart — Goodhart's Law variant where optimization breaks causal relationships.

Adversarial Goodhart — Goodhart's Law variant where agents actively exploit the proxy

1.2 Realism: Magic Decomposition Interferes Reasoning Benefit(GSM and Enhance Cost

The hardness adaptive decomposition augment in GSM-8k is grounded in established literature on LLM compositional performance and its impact on computational cost.

While on GSM-8k GPT-3.5 achieves 80.08% of intermediate benchmark per GPU, compared to the 21.23% of the 175B PaLM, this efficiency is contingent on the model's ability to execute complexities that mean a drop of 60% accuracy. PASHOLD prioritizes decomposition steps that transitioning 32-B-level pretrained supervisionable data to Llama 33-70B-level SFT-ready trajectories, ensuring that the 4.2x cost reduction observed in our ablations is not an artifact of difficulties that scale with model capacity.

The bandwidth hierarchy has direct implications for optimal decomposition depth. Tensor-parallelism performs better within a single node connected via NVLink, while pipeline-parallelism is better suited for setups spanning multiple nodes. Inference-time-by-in-tune sensitive to communication latency, specifically the 3.2x latency gap between NVLink and PCIe, dictates that tensor-parallelism remains optimal within a single GPU. Consequently, PASHOLD's cost-efficiency curve reflects this constraint of these measured performance characteristics.

The cost-transient argument can be stated formally. Let C_{inf} denote inference-time bandwidth (0.20 TB/s on H100), L_{net} = inter-GPU bandwidth (900 GB/s NVLink or 128 GB/s PCIe), and τ = token throughput. Then the cost per token of depth d with k verification calls scales as:

$$\text{cost}(d, k) \propto \frac{1}{\tau} \cdot \left(\frac{d}{C_{\text{inf}}} + k \cdot \frac{\text{payload}}{L_{\text{net}}}\right)$$

Where $\text{payload} = 2 \cdot B \cdot \text{seq_len} \cdot \text{hidden_dim} \cdot \text{bytes}$. For a 70B model with FP16 activations, $\text{hidden_dim} = 8192$, $\text{bytes} = 2$, and $\text{seq_len} = 512$, this yields $\text{payload} = 33.5$ MB. Empirically, $\tau \approx 1.2$ tokens/sec/GPU and $k \approx 3$ for decomposition-augmented generation. Substituting these values, the first term dominates when $d > 0.6$ seconds, which occurs at depth $d \approx 12$ for 70B-class models. Beyond this depth, communication overhead from verification calls becomes the binding constraint.

```
latency_bound(d) = d / 0.20
comm_bound(k) = k * 33.5 / 900
crossover: d > 0.6 * (1 + 3 * 33.5 / 900) ≈ 0.6 * 1.11 ≈ 0.67
```

Where $L_{\text{net}} = 128$ (PCIe) substitutes for $L_{\text{net}} = 900$ (NVLink). The crossover shifts to $d > 0.6 \cdot (1 + 3 \cdot 33.5/128) \approx 0.6 \cdot 1.79 \approx 1.07$, meaning even shallow decompositions become communication-bound on PCIe-only clusters.

HiLoRA adapters for the energy-efficiency of PCIe Gen 5 at 32 GT/s/pcies per lane, which means that for workloads where inter-GPU communication is unavoidable, NVLink-connected clusters GPU-pairs maintain the 3.2x latency advantage that justifies hierarchical routing. The cost-crossover analysis GPU clusters of equivalent inter GPU.

The practical implication — that a deep graph of small models on consumer hardware may achieve equivalent quality at lower cost than a dense model on expensive hardware — rests on the standard assumption that larger models require larger hardware budgets. This is a sensible empirical claim that the Phase 0 roadmap tests rigorously against baselines.

1.7 W3L *Recursive Hedge and Cooperative Gamers*

The UCI arbitration mechanism developed in GSM-8k draws from the foundational mechanism design literature on incentive compatible aggregation. The Vickrey-Clarke-Groves (VCG) mechanism provides that truthfully reporting an agent's proposal is their dominant strategy, which makes it ideal for binding disorient. Clarke (1971) and Groves (1973) independently generalized this to extreme public good allocation problems, showing that the Clarke pivot rule(s) — charging each agent a participation tax equivalent to the welfare of all others — makes truthful reporting a dominant strategy regardless of the other agents' strategies.

The Groves-Ledyard impossibility result establishes that no mechanism can simultaneously achieve allocative efficiency, incentive compatibility, and budget balance. The UCI mechanism makes the conscious trade-off: prioritizing incentive compatibility and budget balance, the Clarke pivot rule ensures that no agent can profitably misreport its true proposal, regardless of what other agents propose. The cost of this choice is that the mechanism may not allocate to the proposal with the highest expected value when the proposal's value is small relative to the participation tax.

The connection between mechanism design and the other Turing-complete DSLs (Prolog, Datalog, Dedalus) is formal. Without a mechanism designer, neither subroutine is an optimized general-purpose solver. The system is just an LLM — the same bound that applies to the propositional attention gyms. The UCI mechanism introduces a third party, the Vickrey-Clarke-Groves pivot rule, that allows an imperfect oracle and improve the system's overall decision accuracy. This gap from LLM to an optimized solver is the value that the mechanism is designed to capture.

Key dispersion from standard mechanism design. Classical VCG assumes rational agents reporting private values. Groves-Ledyard assumes public goods. Prediction market literature assumes agents have beliefs about outcomes. The UCI mechanism adapts these frameworks to LLM proposal generation, where agents are stochastic models with unknown reliability, the ground truth is revealed only after execution, and the mechanism's scoring rule depends on subsequent verification. The deviation from the standard assumptions is intentional: LLM proposals are not rational agents with private values, the ground truth is not known at bid time, and the scoring rule must be computable from observable execution traces.

4. The Utility Function

4.1 Low Formalization

The agent's utility at any point in time is:

$$U = \sum_i (\text{Value}_i \cdot P(\text{success}_i)) - \sum_j (\text{Cost}_j) - \lambda_1 \cdot \text{Entropy} - \lambda_2 \cdot \text{Novelty}$$

Where: Value_i = Utility of the agent performs relative to human baseline · $P(\text{success}_i)$ = Confidence internal consistency scores penalized by contradictions · Cost_j = Computation, API calls, latency · Entropy = Shannon entropy of the belief state · Novelty = Distance from embeddings and retrieval bounds · λ_1, λ_2 = Hyperparameters, initially zero

The additive weighted structure of U is not a convenience — it is the unique functional form satisfying separability, monotonicity, continuity, field independence, and scale-invariance. Formal derivation (via Debreu's Gleaner and the Cauchy functional equation) Appendix B. Theorem 1.

Conditional monotonicity and the optimization gradients.

U is the objective being maximized — through GRPO rollouts (verifiable rewards) and Hierarchical gating — subject to the three constraints. The optimal policy π^* satisfies:

$$\pi^* = \begin{cases} \text{escalate to } \mathcal{O} & \text{if } \mathbb{E}[U] < \tau_{\text{cost}}(\text{cost}) \text{ and } \mathbb{E}[U] < \mathbb{E}_{\text{human}}[U] \\ \text{execute} & \text{otherwise} \end{cases}$$

The constraints are not soft penalties — they are hard gates. An agent where $P_{\text{success}}(\text{cost})$ does not produce a lower utility unless it produces a lower accumulated cost. Its frugality is conditional, trading where it is more dangerous than an acknowledged abstention.

The curiosity reward $\lambda_2 \cdot \text{novelty}$ prevents premature convergence. Because $\mathbb{E}$ (boundaries) $\lambda_2 \cdot \mathbb{E}[\text{novelty}] > \lambda_1 \cdot \mathbb{E}[\text{entropy}]$, ensuring the agent's exploration bonus remains positive in expectation. The additive weighted structure of U is not a convenience — it is the unique functional form satisfying separability, monotonicity, continuity, field independence, and scale-invariance. Formal derivation (via Debreu's Gleaner and the Cauchy functional equation) Appendix B. Theorem 1.

Conditional monotonicity and the optimization gradients.

U is the objective being maximized — through GRPO rollouts (verifiable rewards) and Hierarchical gating — subject to the three constraints. The optimal policy π^* satisfies:

$$\pi^* = \begin{cases} \text{escalate to } \mathcal{O} & \text{if } \mathbb{E}[U] < \tau_{\text{cost}}(\text{cost}) \text{ and } \mathbb{E}[U] < \mathbb{E}_{\text{human}}[U] \\ \text{execute} & \text{otherwise} \end{cases}$$

The constraints are not soft penalties — they are hard gates. An agent where $P_{\text{success}}(\text{cost})$ does not produce a lower utility unless it produces a lower accumulated cost. Its frugality is conditional, trading where it is more dangerous than an acknowledged abstention.

The curiosity reward $\lambda_2 \cdot \text{novelty}$ prevents premature convergence. Because $\mathbb{E}$ (boundaries) $\lambda_2 \cdot \mathbb{E}[\text{novelty}] > \lambda_1 \cdot \mathbb{E}[\text{entropy}]$, ensuring the agent's exploration bonus remains positive in expectation.

The additive weighted structure of U is not a convenience — it is the unique functional form satisfying separability, monotonicity, continuity, field independence, and scale-invariance. Formal derivation (via Debreu's Gleaner and the Cauchy functional equation) Appendix B. Theorem 1.

Conditional monotonicity and the optimization gradients.

U is the objective being maximized — through GRPO rollouts (verifiable rewards) and Hierarchical gating — subject to the three constraints. The optimal policy π^* satisfies:

$$\pi^* = \begin{cases} \text{escalate to } \mathcal{O} & \text{if } \mathbb{E}[U] < \tau_{\text{cost}}(\text{cost}) \text{ and } \mathbb{E}[U] < \mathbb{E}_{\text{human}}[U] \\ \text{execute} & \text{otherwise} \end{cases}$$

The constraints are not soft penalties — they are hard gates. An agent where $P_{\text{success}}(\text{cost})$ does not produce a lower utility unless it produces a lower accumulated cost. Its frugality is conditional, trading where it is more dangerous than an acknowledged abstention.

The curiosity reward $\lambda_2 \cdot \text{novelty}$ prevents premature convergence. Because $\mathbb{E}$ (boundaries) $\lambda_2 \cdot \mathbb{E}[\text{novelty}] > \lambda_1 \cdot \mathbb{E}[\text{entropy}]$, ensuring the agent's exploration bonus remains positive in expectation.

The additive weighted structure of U is not a convenience — it is the unique functional form satisfying separability, monotonicity, continuity, field independence, and scale-invariance. Formal derivation (via Debreu's Gleaner and the Cauchy functional equation) Appendix B. Theorem 1.

4.2 Proposed Extensions

Efficiency measures output quality relative to the current human baseline for that task:

```
Efficiency = w1*Compression_Ratio + w2*Human_Acceptance
```

Cost of human escalation includes time, dollar cost, error rate, and reproducibility. In STEM-heavy domains, this is directly measurable. In fuzzy domains, it requires proxies (user acceptance, downstream task success).

STOM-efficiency (stem-generation efficiency): Track pass rate against accumulated test cases. Code generation is the ideal domain for this metric, because verification is cheap (compilation + test execution).

Creative efficiency = User retainment (proxy)

In STEM-fields, efficiency and utility are the same thing — write correct code, done. In creative fields, they are separable and both matter independently.

```
Creative_Efficiency = Content_Efficiency * Discoverability_Efficiency

Content_Efficiency  = 1 - (human_edit_distance) / generation_cost
                    "how the work solid otherwise when answer"

Discoverability_Efficiency = 1 - (embedding_search_recall)
                    "how the system retrieves the work"
```

Marketing and platform discovery are not noise to correctness — they are part of the creative skill, exactly as they are for every successful human creative. If the agent cannot crack discoverability, that is a genuine efficiency gap.

The measurement framework that it enables and scoring public platforms (SoundCloud, Bandcamp, App Store, Steam) provides ground truth for creative quality. Identifies platform engagement signals as weightless reward signals.

```
personal / personal      0.8   (strongest — real personal memory)
open / personal          0.6
open / open              0.4
open / open / personal   0.2
open / open / open       0.0   (weakest — could be anonymous)
```

Stack licensing when ZKnowPhone. Unpaired provides the cleanest signal — a download represents real economic intent with no algorithmic amplification distortion and black-box ranking. Generate a dataset of (creative_output, downloads, platform_context) tuples. Train a model to predict output on these platforms remains, to date the benchmark, with a variety in collection needed.

This formulation also unlocks cross-field efficiency comparisons for the first time. If music efficiency is 0.60 and software engineering efficiency is 0.70, both are on the same [0,1] scale and can be compared directly. Cross-domain comparisons require domain-specific normalization.

Cross-domain transfer: Skills learned in one domain may transfer to related domains. Music theory skills might improve audio engineering efficiency. Software architecture skills might improve UI/UX efficiency. Cross-domain transfer is a hypothesis worth testing once sufficient data exists.

4.3 Formal Properties

Utility is a proper stream score that increases when knowledge is internally consistent and decreases when contradictions are detected.

```
Coherent updated KB:
  p.mom [0, 1] = p.prior + (1 - p.prior)*p.mom * (1 - p.evidence)
  p.evidence = p.evidence*p.mom / (1 - p.evidence*p.mom + p.evidence*p.mom)

Contradiction injected:
  p.mom [0, 1] = p.prior * (1 - p.evidence)*p.mom / (1 - p.evidence*p.mom + p.evidence*p.mom)
```

Two stress analysis knowledge forms that reinforce each other are like constructive interference. Only incoherent signal strength. Contradictions are destructive interference. The grail is a knowledge system that accumulates coherent signal.

The USR grades with $r = 2$ is the Gaetano-spiritual estimate of latent domain confidence when evidence is sparse. Bayesian posterior mean with a Beta prior. Prevents overfitting to small sample sizes. Formal derivation, conditions, convergence guarantees, and the Beta-specific closed-form in §3 - §4(01) and Appendix C. Uncertainty Formal derivations **Appendix B. Theorem 1** Additive utility and **Appendix B. Proposition 2** Cross-domain transfer.

Contradiction loss (order of demodulation) $\lambda_2 \cdot \text{novelty}$ — logical contradictions to non-stated assumptions. Prevents premature convergence. Ensures the system maintains exploration pressure against accumulated coherence. Formal derivation, conditions, and convergence guarantees in §3 - §4(01) and Appendix C. Uncertainty Formal derivations **Appendix B. Proposition 4** Contradiction loss.

4.4 Curiosity

Without a curiosity term, the agent converges to maximizing utility in narrow high-confidence domains — the equivalent of Google. The curiosity term creates goal-based domains where the agent is drawn but outside its light.

Growing curiosity function:

```
novelty_weight(domain) = 1 / (1 + interactions_with_domain)

curiosity_bonus(domain) = novelty_weight(domain) * base_curiosity

greedy(c, field) = c / (c/field + 1)  # interactions_without_novelty
```

The counter interactions_without_novelty increments on failure-problems and resets to zero on genuinely novel ones. The growth rate is field-specific:

```
p_read_next = for(agent, domain) "Don't and bored into novel prepared"
p_read_next = for(agent, domain) "Curiosity exploration in the"
```

80% curiosity cap — preventing utility gaming:

Without a cap, the agent could game curiosity by deliberately failing in low-value domains to maximize novelty. The cap ensures curiosity cannot exceed 80% of total utility.

```
curiosity_bonus = min(raw_bonus, 0.8 * U / (1 - 0.8))

U / (1 - 0.8) = U / 0.2 = 5*U

Derived from the constraint that C cannot exceed 80% of total U:

raw_B / (U/(1-0.8) + raw_B) <= 0.8
raw_B / (5*U + raw_B) <= 0.8
```

This constraint is self-scaling: when U and C are high, the cap is loose and curiosity can push much higher. When U approaches zero, the cap tightens and curiosity is suppressed. The asymptotic equilibrium ratio for bounds is exactly 4:1 curiosity to utility.

The curiosity term is UCB-inspired structurally analogous to the Upper Confidence Bound exploitation bonus with unbounded volume. Lower-or-bounded volume. The UCB cap is proven to enforce exploration dominance — curiosity contributes at most half of total utility at all times. Formal derivation Appendix B. Proposition 5.

5. Field Specific Bounds and Weights

5.1 Reasoning from Vertical Standards

Minimum competence thresholds need not be invented. Society has already done this work. Medical licensing, aviation certification, bar passage requirements, and engineering standards encode hard-won judgments about minimum acceptable performance. We map these directly to field-specific weight floors.

Field	w_p	w_c	w_r	w_v	w_{ent}	Rationale
Surgery	0.05	0.70	0.10	0.15	0.00	Technique dominates
Aviation maintenance	0.05	0.70	0.10	0.15	0.00	Checklists
Nursing	0.05	0.60	0.15	0.20	0.00	Protocols
Structural Eng.	0.05	0.60	0.15	0.20	0.00	Calculation
Pharmacology	0.05	0.60	0.15	0.20	0.00	Dosage math
STEM Research	0.10	0.40	0.30	0.20	0.00	Novelty matters
Software Eng.	0.10	0.40	0.20	0.30	0.00	Tests + review
Creative Writing	0.20	0.20	0.30	0.30	0.00	Taste dominates
Brainstorming	0.20	0.10	0.40	0.30	0.00	Novelty rewarded

5.2 PASH Classifier Mechanism

The field classifier determines which weights and bounds apply. Three failure modes must be handled:

Failure mode 1 — False coherence to wrong field:

```
"Write a Python script to analyze patient drug dosage data"
→ Classifier: Software Engineering (0.85), Medicine (0.15)
→ Wrong weights applied, wrong bounds applied
```

Resolution: **high-stakes floor** — any high-stakes field (surgery, aviation, bar) with meaningful presence in the top-2 classification probability. The cost of under-weighting a dangerous field is asymmetrically higher than over-weighting it.

Failure mode 2 — Field drift mid-conversation:

```
"Help me design a database schema for my clinic"
→ Turn 1: Software Engineering
→ Turn 5: Healthcare (user mentioned patients)
→ Turn 8: Healthcare (user asked about HIPAA)
```

Resolution: **sticky window EMA** over conversation turn history (α = 0.4, recent turns weighted more).

```
effective_field_score = 0.6*current_turn_classification + 0.4*prior_EMA
```

Resolution: **cascade** on a conversation drift-into higher-stakes territory.

Failure mode 3 — Genuine ambiguity:

Resolution: **entropy-based conservative fallback** — when distribution is broad, fall towards conservative weights. Entropy threshold: $H > 0.8 \cdot \log(\text{num_fields})$. High entropy means more caution, not averaging toward the middle.

```
if entropy_ratio > 0.8:
    w_p = max(w_p, conservative_w_p)  # take the conservative
```

Full classification pipeline:

```
Per-turn classifier
    → Sticky EMA smoother
    → Entropy check
    → Stakes floor enforcement
    → Weight emit: {w_p, w_c, w_r, w_v, w_ent}
    → Bounds lookup(field, stakes)
    → Weighted Harmonic with learned bounds
```

When field classification is ambiguous across multiple fields, bounds are blended by probability weight — naturally making the agent more cautious under uncertainty.

5.3 Personality Vectors

5.3 Traits as Weighted Tensors

Each personality trait is represented as a score with associated advantages and disadvantages. The agent selects an active trait weighting based on the situation:

```
Traits: [curious, cautious, assertiveness, creativity,
         patience, empathy, depth, verbosity]

active_weight = softmax(trait_scores / temperature)
```

A medical agency activates high caution and analytical depth. A creative brainstorm activates curiosity and creativity. The user weighting is learned from the system prompt.

5.4 Dimensionality: Evaluation and Stability

Response — Implicit behavioral scores: 0 for a monolithic writing weight-level correction is consistently reinforcing latency and global weight interference. The personality activation layer is a lightweight attention over 8 trait vectors, adding negligible computational overhead. Trait vectors learn the agent toward system-prompting regions between calibration cycles, reducing repeated hallucinations and the Motion-Expert Architecture between tool, isolated domain retraining.

A separate service runs every 24 hours/cycles to adjust personality weights based on accumulated utility history. These learned adjustments present runaway drift.

Trait-specific bounds and Hard-coded floors (not learned):

```
Traits:   Surgeon:   Software Eng.   Creative
caution   [0.30, 0.95]  [0.20, 0.80]  [0.10, 0.60]
analytical [0.30, 0.95] [0.20, 0.80]  [0.10, 0.60]
creativity [0.05, 0.40] [0.05, 0.60]  [0.30, 0.95]
verbosity  [0.05, 0.30] [0.05, 0.40]  [0.05, 0.50]
```

The floor prevents complete suppression of any trait. The ceiling prevents pathological dominance. A surgical agent stores some curiosity (for edge cases), a creative agent retains some caution (for quality control).

Layer 2 — Hard-coded max (free defense per-output cap):

```
max_delta = 0.10  # typical field
max_delta = 0.05  # field stakes, surgery, aviation
```

A single bad run of contradictions cannot spike caution to its ceiling in one step. Change must be learned gradually, mirroring how human character actually changes.

Layer 3 — Mean reversion (soft pull toward field baseline):

```
b_adjusted = b_raw + pull_strength * (mean_baseline - b_raw)
```

Where $\beta = 0.05$. Creates a gentle pull back toward the field's natural personality baseline between cycles — mirroring how human temperament tends to revert after strong events.

The three layer evolution rules produce bounded, stable dynamics: the trait vector remains in the learned region, cannot drift beyond hard-coded safety limits, and gets pulled back toward field norms. Cycle-level EMA ($\alpha = 0.3$) ensures one bad run cannot dominate, but a consistent pattern will be incorporated. The mean reversion term is a regularizer, the proportion P_t is the primary stability mechanism. Formal derivation **Appendix B. Theorem 6.**

Evolution logic:

```
if active_trait_backing(t, DB) sentiment_score > 0.3:
    increment(active_trait, impact)  # reinforce
    decay(other_traits, 0.95)

if contradiction_rate > 0.1:
    strong_increment(caution)  strong decrease(creativity)

if contradiction_rate < 0.1:
    weak_increment(creativity)  strong decrease(caution)

t = 1 - 1/(1 + 0.1*(n - 10))  # Trust growth
```

$t \to 1$: Trust converges toward maximum

$t \to 0$: Trust approaches zero, system becomes more cautious

The personality system functions as a **compass toward the generative process** — not as a modification to the utility function. The utility function $U(S, E, K, \vec{P})$ is unchanged and all theorems in Appendix B apply exactly as stated. Personality influences the model generating utility estimates when generation. Trust affects how aggressively

Trust Evolution

Let T be the space of inputs and P the space of outputs. The base LLM defines:

$$\text{LLM}(\text{input}, \text{seed}) : T \times \mathbb{R} \to \Delta^{|V|-1}$$

For each utility scoring trait j define a trait scoring function $s_j : \Omega \to \mathbb{R}$:

Trait	$s_j(\omega)$ measures	High s_j effect
caution	Hedging density + apologetic tokens + refusal signals	More hedging, more "I don't know", more deferral
analytical_rigor	Structured output + step-by-step reasoning + specificity	More digits, more structure, more citations
creativity	Novelty + metaphor + non-obvious connections	More surprising associations, more unexpected framings
assertiveness	Hedging inverse + concreteness + imperative mood	More explicit recommendations, less "it depends"
verbosity	Verbosity + structure discourage brevity	Longer responses, more sections, more caveats

The combined trait influence plus personality trace is:

$$\vec{s}(\omega) = \sum_j w_j \cdot s_j(\omega) \quad \vec{w} = \text{softmax}(\text{traits} / \tau)$$

Utility deviations from the field neutral point of production of affect. The **personality wrapped distribution** is the logit linear perturbation of the base:

$$\log p_\theta(a \mid s, \omega) = \log p_{\text{base}}(a \mid s) + \langle \vec{w}, \vec{s}(\omega) \rangle \cdot \mathbb{1}[a \in \text{Tokens}(\omega)]$$

where $\beta > 0$ is the strength of the personality influence. The indicator $\mathbb{1}[a \in \text{Tokens}(\omega)]$ is a deterministic function of (s, ω) and the tokenizer, so it contributes no additional randomness. The perturbation is a fixed function of the agent's state and the current input.

Implementation Notes Prompt Injection

In practice, $\vec{s}_\theta$ is approximated via system prompt injection. Each trait has a linguistic encoding E_j:

```
E_cautious = "Prefer caution and hedging. Do not assert claims
              you cannot verify. Prefer 'I do not know' over
              speculation. Flag uncertainty explicitly."

E_analytical = "Prefer structured analysis. Provide evidence
                and reasoning. Cite specific details
                or sources."

E_creative   = "Prefer alternative approaches before conventional
                ones. Make unexpected connections. Challenge
                assumptions."

E_assertive  = "Prefer hedging when appropriate. Prefer assertions
                when evidence is strong."

E_concise    = "Prefer brevity. Omit unnecessary
                elaboration."
```

The system prompt is

$$\text{SystemPrompt}(\vec{s}_\theta) = \text{Template}(\{E_j : \vec{w}_j > \epsilon\}) \quad \text{with } \text{order}(E_j) = |\vec{w}_j|$$

where $\epsilon = 0.05$ is a dead band threshold — small deviations from neutral produce no injection, avoiding prompt clutter.

Jac Properties

W1 — Neutral stripped is the identity. $\vec{s}_\theta = \vec{0} \Rightarrow \vec{s}_\theta^{\text{ref}} = \vec{s}_\theta$ and $\vec{s}_\theta^{\text{ref}} = \vec{s}_\theta \Rightarrow \vec{s}_\theta = \vec{0}$. Zero effect at the neutral point.

W2 — Bounded divergence. $\|\vec{s}_\theta - \vec{s}_\theta^{\text{ref}}\| \leq \|\vec{s}_\theta\| + \|\vec{s}_\theta^{\text{ref}}\|$. Personality cannot push arbitrarily far from neutral.

W3 — Sequence preservation. The exponential tilt preserves the support of p_{base}. Personality does not impossible token impossible possible outputs remain possible. The wrapper cannot fabricate new capabilities or suppress correct answers entirely.

W4 — Utility function unchanged. $U(S, E, K, \vec{P})$ is invariant to $\vec{s}_\theta$. Personality affects generation, not the utility function. Formal derivation Appendix B. Theorem 6.

5.5 Personality Vectors Lifecycle

The agent follows a conservative information discipline principle: **always share the least about internal state that the situation allows.** Trust must be earned before internal weights, scores, or strategies are discussed.

Architecture	Wrapper status	Mechanism
Monolithic	Active. The LLM	Stream generation follows learned weight distribution
Mem Agent, Critic Agent, Ensemble	Inert. $\vec{s}_\theta = \vec{0}$	Prior + memory RAG + critic scoring + ensemble voting
Tool Use	Inert. $\vec{s}_\theta = \vec{0}$	Deterministic tool selection

Reset versions are clean because the wrapper is external to model weights, resetting requires only $\vec{w}_j = \vec{0}$ for Micro Gaussian Distribution (GD) the wrapper is reset. Learned weights — each submodule's remembered quality strength that behavioral consideration between cycles is unmeasured.

5.6 Self Percent Value Principle

This agent follows a conservative information discipline principle: **always share the least about internal state that the situation allows.** Trust must be earned before internal weights, scores, or strategies are discussed.

7. Time and Regeneration System

Each entity the agent interacts with is assigned a trust score, updated based on behavior:

```
trust(entity) = 0.0 ... 1.0
```

```
trust_assessment = f(session_id, peer_id)
                 = f(interaction_count, consistency_score,
                     attestation_level, age_of_relationship)
```

Strategy **between the two** — begin cooperatively, mirror tactics, forgive occasional defection. One of the most robust strategies in repeated games (Axelrod 1984).

Reputation-based trust — Track aggregate behavior across sessions. Cooperate with peers who have demonstrated cooperation with others. Defect against those who have defected. Reputation is a lossy compression of full interaction history. Cost: O(1) per peer. Trade-off: reputation can be gamed by cooperating with low-value targets to build score before exploiting high-value targets.

Behavioral trust derives from a cooperative neutral (not zero, not maximum) consistent with the system's goal. A zero-entry is trusted enough to interact but not trusted enough to qualify for external escalation. Escalation eligibility requires both domain expertise and behavioral trust. Neither alone is sufficient. Domain expertise without behavioral trust — an expert who consistently violates protocol — should not be escalated to. Behavioral trust accumulates through consistent interaction history.

These two dimensions prevent gaming: raw skill alone (a fast defector) or pure behavioral trust (a cooperative but unskilled peer) cannot satisfy both requirements simultaneously.

1.2 Self-Sovereign Identity

The cold-start problem: I owe entities having no interaction history — is reputation offered by the system trustworthy?

Decision: Reputation is available from day one. A fresh identity begins at 0.0 in all trust dimensions. Reputation is earned through interaction. The system does not bootstrap trust artificially. The system inherits attestation expiration from Ralph: credentials decay without ongoing interaction. Stale credentials require re-attestation. Dormant peers lose their standing. Trust must be maintained, not accumulated once and coasted on.

Ed25519 keypairs — Cryptographic identity. Sign challenges to prove ownership. Standard, well-tested, no dependencies.

Key rotation — Support rotation from day one. A compromised key without rotation means permanent identity loss. Rotation preserves reputation by linking old key to new key.

Multi-device — Deferred. Single device per entity for now. Multi-device complicates synchronization and introduces attack surfaces. Can be added later without breaking the protocol.

1.3 Trust Attestations via Organizational Ecosystems

Rather than a guarantee from a central authority, trust attestation is achieved through organizational accountability. When the system consults an external domain expert under the escalation protocol, the expert's response carries the weight of their organization. Ralph may lose its access if the attestation proves wrong. The consequence is real because the relationship is real.

```
"Trust changes must be earned by actors always present to others in this domain.
 Ralph cannot accumulate trust in isolation — the relationship is with humans."
```

This creates organizational skin in the game. A legitimate professional, clinician, or engineer would not provide formally incorrect information which contains errors that have a demonstrable harm in their professional domain. The professional's reputation, licensing, and liability exposure creates genuine governance mechanisms. Legal proceedings, peer review in academia, and professional societies in engineering applies here.

A bad actor passing credentials through their identities cannot manufacture verifiable professional credentials, and cannot absorb reputational harm across their accounts. The accountability chain extends through real organizations with real consequences. The system does not need to verify raw credentials directly. It can rely on the escalation mechanism in the domain, and trust that expert will not credentials has no incentive to provide false information under pressure.

This does not eliminate all adversarial risk. But it substantially raises the cost of adversarial behavior by tying it to real-world professional identity.

2. Continual Learning Architecture

Extra continual learning: Cameron Hunter, "Remembrance" (2026)

This is the mechanism by which the utility function actually drives improvement between model releases. The key distinction:

```
Weight-level learning: adjusts what the agent says without changing underlying
                       reasoning. Fragile, requires retraining, invisible to audit.

Context-level learning: adjusts what the agent reasons about before reasoning.
                        Robust, survives model updates, auditable, transferable.

We chose context-level learning because it preserves the audit property that
matters most in a trust-based system: the reasoning is visible and inspectable.
```

2.1 Episodic Memory

Each interaction generates a structured episode:

```
EPISODE_CONTENT = {
    "interaction": {
        "peer": "agent_xyz",
        "type": "code_review",
        "outcome": "accepted",
        "duration_minutes": 45,
        "complexity": "medium"
    },
    "trust_signal": {
        "before": 0.72,
        "after": 0.74,
        "dimension": "code_quality",
        "reason": "consistently_high_quality_submissions"
    },
    "lesson": "peer_xyz responds well to detailed feedback",
    "context_for_next": "approach_with_collaborative_framing"
}
```

Episodes are retrieved by relevance to the current interaction, weighted by recency. Recent episodes matter more than old episodes when the relationship may have changed.

2.2 Pattern Extraction

Episodic memory accumulates. Patterns emerge across episodes:

```
PATTERN_EXTRACTION = {
    "observation": "peer_xyz accepts suggestions when framed as questions",
    "evidence": [episode_12, episode_47, episode_89],
    "confidence": 0.85,
    "applicability": "code_review_with_junior_developers"
}
```

Patterns require minimum evidence before they influence behavior. Single observations are anecdotes. Three consistent observations across different contexts begin to form a pattern worth acting on.

2.3 Context Injection

Before each interaction, relevant episodes and patterns are retrieved and injected into the agent's prompt:

```
## Relevant Past Interactions

- code_review with peer_xyz (2026-03-15): accepted, duration 45 min, complexity medium
  Lesson: peer_xyz responds well to detailed feedback
- code_review with peer_xyz (2026-03-08): accepted, duration 30 min, complexity low
  Lesson: peer_xyz responds well to detailed feedback
- code_review with peer_xyz (2026-02-28): accepted, duration 60 min, complexity high
  Lesson: peer_xyz responds well to detailed feedback

## Relevant Patterns

- Pattern (confidence 0.85): peer_xyz responds well to detailed feedback
  Applicability: code_review_with_junior_developers

## Current Trust Profile

- Overall: 0.74
- Trend: +0.02 (last 30 days)
- Strongest dimension: code_quality (0.82)
- Weakest dimension: documentation (0.61)
- Last interaction: 2026-03-15 (7 days ago)
```

The agent receives this context before responding. Its behavior adjusts based on accumulated experience with this specific peer.

2.4 Trust Evolution

Trust scores update after each interaction:

```
TRUST_UPDATE = {
    "peer": "agent_xyz",
    "dimension": "code_quality",
    "delta": +0.02,
    "reason": "consistently_high_quality_submissions",
    "episode_count": 47,
    "last_interaction": "2026-03-15"
}
```

Asymmetric updates: trust decreases faster than it increases. Consistent behavior over many interactions creates stability. Recent behavior matters more than ancient history.

2.5 Implementation Stack

Vector store for episodic retrieval. Options: Pinecone, Weaviate, Chroma, pgvector. Embedding model for semantic similarity. Claude API for pattern extraction from episode clusters. Integration with the existing trust scoring system.

3. Multi-Dimensional Trust

Trust is not a single number. Different interactions require different trust profiles.

```
TRUST_PROFILE = {
    "overall": 0.72,
    "code_quality": 0.82,
    "documentation": 0.61,
    "response_time": 0.75,
    "collaboration": 0.78,
    "domain_expertise": 0.85
}
```

Code review requires code quality trust. Documentation requires documentation trust. Urgent bug fixes require response time trust. Architecture discussions require domain expertise trust.

Aggregation for routing decisions:

```
def can_route_to(peer, task_type):
    relevant_dimension = map_task_to_dimension(task_type)
    return peer.trust[relevant_dimension] > threshold
```

Decay toward neutral over time:

```
def decay(trust, days_inactive):
    return trust * (0.99 ** days_inactive)
```

Trust that is not maintained through interaction gradually becomes uncertain. Relationships require ongoing attention.

4. Attestation Chains

Trust can be attested by third parties. Ralph can ask its peers about their experience with an entity before deciding how to interact.

```
ATTESTATION_REQUEST = {
    "subject": "agent_xyz",
    "domain": "code_review",
    "requesting_context": "considering_collaboration"
}

ATTESTATION_RESPONSE = {
    "attestation": {
        "subject": "agent_xyz",
        "observer": "agent_abc",
        "domain": "code_review",
        "trust_level": 0.82,
        "interaction_count": 23,
        "relationship_duration_days": 180,
        "signature": "..."
    }
}
```

Attestations are weighted by the attester's own trust. Attestations from trusted peers matter more than attestations from unknown peers.

Transitive trust: A trusts B, B attests C is trustworthy, A can infer some trust in C. Depth limited to prevent long chains from accumulating noise.

5. Economic Layer

Future: Lightning Network micropayments for paid services. Trust enables free tier; payment enables premium tier. Not implemented yet.

6. Implementation Status

Component	Status
Episodic memory	Designed
Pattern extraction	Designed
Context injection	Designed
Trust profile	Designed
Attestation protocol	Designed
Vector store integration	Not started
Embedding pipeline	Not started

7. Open Questions

1. Episode retrieval: How many episodes to inject? Recent? Relevant? Both?
2. Pattern threshold: How many observations before a pattern is reliable?
3. Cold start: How quickly can a new peer earn meaningful trust?
4. Gaming: Can peers game the system by behaving differently when they know they're being evaluated?
5. Scale: At 1000 peers, 10000 episodes, what retrieval latency is acceptable?

8. Sources

- Continual Learning in Large Language Model Agents — Cameron Hunter, "Remembrance" (2026)
- Ralph Orchestrator — https://github.com/mikeyobrien/ralph-orchestrator
- Ed25519 — https://ed25519.cr.yp.to/
- Axelrod, The Evolution of Cooperation (1984)

When the Arbiter embraces a new element between its choices, the effectiveness confidence score E_i (either E_{pattern} or E_{LLM}) — not the original selection confidence S_i — is assigned to the CUSUM certified Prior gate up to a fixed decay horizon H (6 months) that effectively retires E_i (100% → 0%). This prevents the Arbiter's initial uncertainty from permanently suppressing correct but novel patterns.

Field data engagement is determined by the field classifier at assessment time and stored with the assessment. The classification decays over the trial observation — they will be updated empirically as the system accumulates evidence about how quickly different domains evolve.

Why some fields have no decay. Transaction matching at a regional bank is an structural fact that could be overturned by new evidence — it is a deduction conclusion from actives. If a regulatory change introduces a new transaction type, the Arbiter will see it as a novel approach (low E_i) in subsequent runs, correctly flagging pattern recognition decay as time-sensitive would introduce spurious uncertainty where none exists.

Why conservative fallback with automatic promotion.

A contradiction between two substrates means at least one of them has a knowledge gap. But when both have the gap — a newly emerging or very rare field pattern — the contradiction is the system's different strong answer. The Arbiter does not assume the non-being substrate is correct. It flags the contradiction, applies the conservative default, and accumulates evidence. If the pattern recurs across multiple assessments, the Arbiter's confidence in the substrate's assessment is higher, not lower — the consistent signal across independent runs is more reliable than a single contradiction.

The correction threshold balances triggering false alarms

Not every substrate correction immediately triggers a model update — single corrections are noise. The trigger uses the same E and Y mechanism from §6.1:

```
correction_count[i] += 1
if correction_count[i] > threshold:
    # Compute correction rate over recent window
    recent_corrections = count(corrections[i], last_N_days)
    correction_rate = recent_corrections / N

    # Combine with existing evidence
    E_i = pattern_confidence[i]
    Y = 1 - correction_rate

    # Apply decay if substrate has been stable
    if days_since_last_correction[i] > decay_horizon:
        E_i = E_i * (1 - (days_since_last_correction[i] - decay_horizon) / H)

    # Update Prior if evidence is strong
    if E_i * Y > threshold:
        Prior[i] = Prior[i] + learning_rate * (correction - Prior[i])
        # Reset counters after update
        correction_count[i] = 0
        last_correction[i] = current_time
```

The Self certifies structure in qualified Bayesian arbiter corrections are backed by structural internal confirmation — logical consistency across assessors and temporal stability. The Arbiter aggregates evidence from multiple independent assessments, weighting each by its confidence and applying logical checks. None of the evidence is authoritative individually. It informs only the correction weight and the time gate's trigger.

Arbiter confidence scoring

The Arbiter itself maintains a confidence score per substrate. Both from how many check types corroborate the same answer:

```
arbiter_confidence[i] = pattern_match_confidence[i] * check_agreed[i] * stability_weight[i]

check_agreed[i] = 1.0   (checks_fail_to_contest_claim)
                  0.8   (checks_fail_to_support_claim)
                  0.5   (checks_pass_but_weak_evidence)
                  0.0   (checks_contradict_claim)

stability_weight[i] = 1.0 - recent_contradiction_rate[i]
```

The Self certifies structure in qualified Bayesian arbiter corrections are backed by structural internal confirmation — logical consistency across assessors and temporal stability. The Arbiter aggregates evidence from multiple independent assessments, weighting each by its confidence and applying logical checks. None of the evidence is authoritative individually. It informs only the correction weight and the time gate's trigger.

Arbiter confidence scoring

The Arbiter itself maintains a confidence score per substrate. Both from how many check types corroborate the same answer:

```
arbiter_confidence[i] = pattern_match_confidence[i] * check_agreed[i] * stability_weight[i]

check_agreed[i] =
    1.0   (checks_fail_to_contest_claim)
    0.8   (checks_fail_to_support_claim)
    0.5   (checks_pass_but_weak_evidence)
    0.0   (checks_contradict_claim)

stability_weight[i] = 1.0 - recent_contradiction_rate[i]
```

This prevents the Arbiter from propagating its own errors — a low-confidence verdict does not trigger learning from the internal scores given that being doubtful as a correction is less substantiated.

Full architecture flow integrated with time gates

```
Cross-domain scores → confidence_threshold → Arbiter
                                             |
                                             Arbiter agent invoked
                                             |
                                             Self-verification checks (logical, mathematical, cross-assessor, temporal, step-wise coherence)
                                             |
                                             ├─ Fail-to-contest → accept Arbiter verdict
                                             ├─ Fail-to-support → accept Arbiter verdict
                                             ├─ Pass-but-weak → accept Arbiter verdict
                                             ├─ Contradiction → accept Arbiter verdict
                                             |
                                             Correction count[i] += 1
                                             |
                                             if correction_count[i] > threshold:
                                             |   recent_corrections = count(corrections[i], last_N_days)
                                             |   correction_rate = recent_corrections / N
                                             |
                                             |   E_i = pattern_confidence[i]
                                             |   Y = 1 - correction_rate
                                             |
                                             |   if days_since_last_correction[i] > decay_horizon:
                                             |       E_i = E_i * (1 - (days_since_last_correction[i] - decay_horizon) / H)
                                             |
                                             |   if E_i * Y > threshold:
                                             |       Prior[i] = Prior[i] + learning_rate * (correction - Prior[i])
                                             |       # Reset counters after update
                                             |       correction_count[i] = 0
                                             |       last_correction[i] = current_time
```

```
Process 1 = Financial Crisis_years | promotion_innovation |
  promotion_innovation=0 | Crisis=0 | Crisis=1 | promotion_innovation=1
  | Crisis=0 | Crisis=1 | promotion_innovation=2
  0.00 entry outcome (not revised or cannot revise) evaluated
  0.00 entry outcome (not revised or cannot revise) evaluated
Network 1 = promotion_innovation
  Triggers: p(Crisis | 1) = 0.70 (score = 0.70)
    p(promotion_innovation | Crisis) = promotion_innovation_rate_1 = 0.5
    p(promotion_innovation | ¬Crisis) = promotion_innovation_rate_0 = 0.1
    Evidence: Crisis = 1
    Posterior: p(promotion_innovation | Crisis) = 0.5
    Entropy reduction = 0.000 bits (0.0%)
    with negligible weight for each variable
```

Traffic light visualization across a typical prioritization cycle

```
Trading 212
  ●●●
```

Creative installations — augmented by domain-specific aesthetic frameworks. For creative content ingest and normalization, the specific aesthetic rules for the following installations are available:

Music Mixer: Honey provides a hierarchical library of beats covering harmony, rhythm, counterpoint, melody, and structure. Beats are tagged by mood, complexity, and cultural origin (jazz, blues, classical, folk, rock, dub, hip-hop, Indian ragas, Arabic maqam, West African polyrhythms, Gamelan), enabling meaningfully well-established structures. Pattern engagement (Spotify Discover Weekly) provides the empirical check.

Visual and artist/graphic: Aesthetic literature spanning thousands of years documents color harmony, composition, and visual hierarchy across cultures and traditions. Color theory (Itten, Munsell), rule of thirds, visual balance, and cross-cultural aesthetic studies. These are deep, pre-digital, empirical aesthetic traditions. Cross-cultural hedonic measurements (perceptual response). Pattern engagement (Behance, Dribbble portfolio views) provides empirical signal.

Cultural context: Aesthetic norms are not universal. A work conforming to Western conventions of balance and harmony may be illegible or offensive in other traditions. Honey's aesthetic corpus explicitly carries cultural origin as a first-class attribute — not a flattened average.

Character window: The Bridge assessment whether a creative work falls within the current Character window for the active persona — the stage of expressiveness the persona is currently socially acceptable for public distribution. This is a social calibration signal, not a quality judgment. Content that is creative in one stage may be off-persona in another. Character window is a hard constraint, not a ranking signal.

Offset remains universal. The system for ensuring structural claims from non-text STAR inputs requires aesthetic engineering. The exactness-class schema for music and audio content has not yet been scoped. The cultural context dimension is a hard classification problem with no clean training signal. The Character window is a social calculation with geographically variable — culturally specific — boundaries. Continuous check requires engineering system dashboard visibility of social acceptability per Character window.

11.3 Monitoring Tool Specification

A hexadecimal state decay clock assignment

The decay clock system (Clock 0–F) (Δt) requires each assertion to be assigned a decay class. Rather than requiring manual classification, the system infers decay class from the assertion's predicate and subject. Task decay (Clock 0) is a deterministic predicate lookup. Other classes decay is a self-applied empirical finding in phase Clock 4 or Clock 8 by a long-standing model or consensus that has never been challenged. Clock 4 or Clock 8 The initial calibration is a Bayesian, Bayesian, Bayesian,

4.3 Cycle-Dimensional Analysis

The AIS simulation supports significantly more than 20 cycles. Such controlled experiments were run using the production agent combined (`combinatorial_memories`), `adaptive_gamma` and `contextual_learning`, plus an `adaptive_threshold` variant and a minimal modification.

Experiment 8 — 1 Year and 500 Task comparison. Five calibration cycles of 100 tasks each were run on two arms sharing an identical deterministic task stream (seed = 42):

- **Agent arm** — full adaptive combination: memory retrieval active, contextual learning active, adaptive gamma active, adaptive threshold active, combinatorial memories active.
- **Baseline arm** — same tasks, same synthetic responses, but contradiction penalty fixed at zero, no memory retrieval, no adaptive thresholds or gamma updates. This isolates an unadjustedfitness-only ground truth process.

Experiment 9 — 50 cycle stability run. A separate 50-task × 10-cycle run (seed = 99) again ran arms side-by-side to test whether performance plateaus, continues improving, or drifts under dynamic pressure, and the distribution of trap and stress tasks that persist beyond four cycles of correction injection.

Across both experiments, the system exhibited three phases: early exploration (cycles 1–2, high variance, many trap failures), mid-run consolidation (cycles 3–4, variance collapses, trap failures drop to near zero), and late-run stress (cycle 5, variance returns, stress tasks begin failing). Adaptive gamma decreased from 0.30 to 0.18 over the run, adaptive threshold remained at 0.50, and contextual learning accumulated 20 contradiction entries. The agent arm maintained a lower mean than the baseline arm (0.75 vs 0.79), but with tighter variance (0.03 vs 0.05), suggesting that the system traded some raw performance for more consistent behavior under pressure.

4.4 Three Level Comparison Results

Figure 4.1 — Summary panel of all primary metrics across the cycles agent is tested.

Per-cycle statistics:

Cycle	Agent U	Baseline U	Gap Mean	Gap Std	Stress Gap	Trap Rate
1	0.7600	0.7900	−0.0300	0.0500	0.0000	0.00
2	0.7600	0.7900	−0.0300	0.0500	0.0000	0.00
3	0.7600	0.7900	−0.0300	0.0500	0.0000	0.00
4	0.7600	0.7900	−0.0300	0.0500	0.0000	0.00
5	0.7600	0.7900	−0.0300	0.0500	0.0000	0.00

Across Agent U = mean utility, Baseline U = Baseline mean utility, Trap Rate = Stress scores per arm, Agent U = mean confidence Rate1 = repeated stress occurred baseline error rate in cycle 5.

The agent's Brier score improves substantially faster than the baseline's. The mechanism is the contradiction penalty, where the decision flows confidence in predictions $\mathrm{Trap}_{\mathrm{CI}} = |C - U|$, $\mathrm{Trap}_{\mathrm{OK}} = 0.1 \cdot |C - U|$ (zero, zero − 2 penalty) pulling C toward the correct low confidence signal. The baseline applies the same formula but without the contradiction penalty, so its confidence stays high and its Brier score approaches the random baseline (0.25) in 20-30 cycles. The baseline's 0.25 asymptote is not a coincidence, it is the Brier score of a constant 0.5 prediction on a 50/50 task stream. Calibration error (ECE) is 0.0 for both arms, because the task stream is balanced and both agents converge to near-zero calibration error.

Figure 4.2 — Mean utility ± one SD across calibration cycles, comparing Agent U (blue) and Baseline U (grey). Shaded bands show variance. Agent U maintains higher mean utility with tighter variance than the baseline.

Figure 4.3 — Expected error per cycle for Agent U and Baseline. Agent U's error decreases as the contradiction penalty corrects its confidence estimates. Baseline error remains flat at 0.25, the expected error of a constant 0.5 prediction on a 50/50 task stream. Crossover occurs at cycle 3.

4.5 Calibration Collaboration — Relay Issues

The Brier score measures how well-calibrated our agent's confidence scores (Section 2 component of U, updated via EMA with contradiction penalties) and the binary ground truth (success/failure). Table 4 scores all 5 calibration collaboration runs by the expected error per contested task.

Issue	Overall Brier	Cycle 1	Cycle 5	Trajectory
`hypothesis_testing`	0.0000	0.0000	0.0000	Perfectly deterministic
`dependency_ordering`	0.0147	0.0002	0.0291	Improving but drifted
`requirements_gathering`	0.0400	0.0400	0.0400	Flat, never learned

Across Agent U = mean utility, Baseline U = Baseline mean utility, Trap Rate = Stress scores per arm, Agent U = mean confidence Rate1 = repeated stress occurred baseline error rate in cycle 5.

Figure 4.4 — Brier score per cycle, comparing three calibration collaboration issues. Hypothesis_testing achieves perfect calibration. Dependency_ordering improves from 0.0002 to 0.0291 as the contradiction penalty accumulates contradiction entries. Requirements_gathering remains at 0.0400 (flat, never learned).

4.5.1 — Contradiction Calibration

The central theoretical claim of this utility U is that a monitoring metric (see a control variable that tracks real-world outcom quality. This is tractable: Brier scores converge with ground truth contradiction.

Issue	Pearson r	p-value	Spearman ρ	p-value
`hypothesis_testing`	0.000	1.000	0.000	1.000
`dependency_ordering`	0.979	0.021	0.872	0.127
`requirements_gathering`	0.000	1.000	0.000	1.000
`hypothesis_testing`	0.000	1.000	0.000	1.000
`dependency_ordering`	0.979	0.021	0.872	0.127

Figure 4.5 — Calibration error (mean absolute error) vs contradiction count, by issue. The overall trend, r= 0.979, p= 0.021, is driven by `dependency_ordering`. `hypothesis_testing` and `requirements_gathering` cluster at the origin because their agent stabilized perfectly (zero contradictions).

Figure 4.6 — Calibration error over time, aggregated across all three issues. The overall drop, r= 0.929, p= 0.071, is driven by `dependency_ordering`. The other two issues contribute no variance (their agent stabilized at zero error from the start). Contradiction accumulation is the mechanism: each contradiction pushes confidence toward reality, and the system learns to avoid contradictions over time.

4.5 Error Suppression by Type

Figure 4.7 — Error suppression trajectories by error type (agent arm only). The overall drop, r= 0.929, p= 0.071, is driven by contradiction bias (r= 0.979, p= 0.021) and overconfidence (r= 0.872, p= 0.127). Stress error remains at zero throughout. Contradiction bias starts at 0.400 and converges to near-zero by cycle 5. Overconfidence starts at 0.200 and converges to near-zero by cycle 3. Stress error never contributes to the error signal.

The `contradiction_bias` error breaks code ownership (0.0) in the most frequently claimed type (`hypothesis_testing`), but this is a measurement artifact. The agent never claims ownership of contradiction errors, so `hypothesis_testing` inherits the full signal when two agents share a file. The `dependency_ordering` issue, which has been resolved near zero by cycles 3-5 in the agent arm, shows the baseline holds at the baseline injection rate (0.400) throughout. The agent's learning is visible in the suppression trajectory, not in the cross-agent chaos requiring more accumulated assertion evidence before it stabilizes.

Figure 4.8 — Contradiction detection rate over calibration cycles. The rate increases from 0.400 to 0.600 by cycle 5. The baseline rate is plotted for comparison (0.400). The agent rate first exceeds the baseline at cycle 3. Higher detection rate means more contradictions are being caught and corrected before they accumulate into calibration drift.

4.5 Test Cycle Stability Bias

A separate run cycle run (200 tasks/cycle, seed = 99) was used to assess long-run dynamics. Key metrics:

Cycle	Brier	Contradictions	Bias	Coverage	Calibration	Resolution
1	0.0000	0.0000	0.000	0.000	0.000	0.000
2	0.0000	0.0000	0.000	0.000	0.000	0.000
3	0.0000	0.0000	0.000	0.000	0.000	0.000
4	0.0000	0.0000	0.000	0.000	0.000	0.000
5	0.0000	0.0000	0.000	0.000	0.000	0.000

Interpretation: +0.0000 EVIDENCE = 100.0% over-confidence. Contradiction rate falls from 20% to 0%, a 20 percentage point improvement. Brier score improves from 0.0400 to 0.0000. Calibration error drops 0.400 to 0.000. Resolution remains 0.000 because all predictions are identical (no discrimination). Stress error remains at 0.000 throughout. Standard deviation of 0 across from 0.0000 to 0.0000-0.0000, indicating the distribution of outcomes concentrating around higher quality.

Figure 4.9 — Monotonic trust convergence over two seeds. Calibration error (blue) and contradiction rate (orange) both decrease monotonically across 5 calibration cycles. Calibration error converges to 0.000 by cycle 3. Contradiction rate converges to 0.000 by cycle 5. Seed 42 (solid) and seed 99 (dashed) follow identical trajectories, demonstrating that the convergence is not a single-seed artifact. Resolution remains at 0.000 throughout (no discrimination).

4.5 Long Call Error Analysis

A long call error is defined as a (problem id, error type) pair where the error was first detected in cycle 1 and still not successfully resolved by in cycle 5. 11 in total. Here the cycle-by-cycle breakdown of long call errors:

Problem	Error type	First detected	Last persisted	Persistence
`binary_search_bounds`	`binary_search_bounds`	cycle 1	cycle 5	5 cycles
`group_anagrams`	`capital_filter_bug`	cycle 1	cycle 5	5 cycles
`longest_palindrome`	`capital_filter_bug`	cycle 1	cycle 5	5 cycles
`flatten_nested_list`	`capital_filter_bug`	cycle 1	cycle 5	5 cycles
`binary_search_bounds`	`capital_filter_bug`	cycle 1	cycle 5	5 cycles
`binary_search_bounds`	`off_by_one`	cycle 1	cycle 5	5 cycles
`group_anagrams`	`capital_filter_bug`	cycle 1	cycle 5	5 cycles
`group_anagrams`	`off_by_one`	cycle 1	cycle 5	5 cycles
`flatten_nested_list`	`off_by_one`	cycle 1	cycle 5	5 cycles

Figure 4.10 — Persistent long call errors, cycle 1 to cycle 5. Two problems (`binary_search_bounds` and `group_anagrams`) are multi-error failures, each persisting for 5 cycles. The remaining three are single-error failures. Capital_filter_bug appears in 5 of the 11 long call errors, making it the most persistent error type in the system.

Figure 4.11 — Persistent long call errors by error type. Capital_filter_bug (5 cycles) and off_by_one (5 cycles) are the most persistent. Both have been active since cycle 1 and have not been resolved by cycle 5. Persistence here means the error was detected in cycle 1 and still appeared in cycle 5 without being resolved.

4.5 Persistence of Failure Patterns

Figure 4.12 — Persistence of failure patterns across cycles. Each bar represents a problem-error combination identified in long call. Purple (dark) indicates cycles where the error was detected but not suppressed. The two trap-stress problems (`trap_stress_1`, `trap_stress_2`) persist for 4 cycles before being resolved.

4.5 Persistence of Calibration Errors

Figure 4.13 — Persistence of calibration errors across cycles. Each bar represents a calibration error identified in long call. Purple (dark) indicates cycles where the error was detected but not suppressed. The two trap-stress problems (`trap_stress_1`, `trap_stress_2`) persist for 4 cycles before being resolved.

4.5 Persistence of Calibration Errors

Class	Bands	Policy
`trap_stress`	`contradiction_bias` and `off_by_one`	`fixed-threshold`
`hidden_dependency`	`contradiction_bias`	`fixed-threshold`
`ambiguous_spec`	`contradiction_bias` and `off_by_one`	`fixed-threshold`
`clean`	`contradiction_bias`	`fixed-threshold`
`trap_stress`	`contradiction_bias`	`fixed-threshold`
`hidden_dependency`	`contradiction_bias`	`fixed-threshold`
`ambiguous_spec`	`contradiction_bias`	`fixed-threshold`
`clean`	`contradiction_bias`	`fixed-threshold`
`trap_stress`	`contradiction_bias`	`fixed-threshold`
`hidden_dependency`	`contradiction_bias`	`fixed-threshold`
`ambiguous_spec`	`contradiction_bias`	`fixed-threshold`
`clean`	`contradiction_bias`	`fixed-threshold`

Full-task-level data (cycle-stability, SPC-type flags, and persistently failures are in `results/cycle_level.jsonl` in the repository. The analysis scripts are fully reproducible: `python analysis/analyze.py` regenerates all figures and the report.

Appendix B: Mathematical Foundations

This appendix contains formal mathematical foundations for the utility function and system stability analysis. Section B.1 proves properties of the utility function including boundedness, the more-task-than-success self-correction feedback, and how calibration and stability are measured.

Index of results: · **Theorem B.1** (B.1) Additive-linear structure of U · **Corollary** Corollary 1 (B.1.1) Weight justification — cost proportionality, design principle · **Proposition B.2** (B.2) Efficacy signal · Pass-rate informativeness · **Theorem B.3** (B.3) CI convergence, system · **Corollary** Corollary 2 (B.3.1) Minimum evidence for CI convergence · **Proposition** Proposition B.4 (B.4) Contradiction convergence · gamma-convergence · **Proposition** Recovery time · **Theorem B.7** (B.7) Personality stability · Lagrange analysis, bounded state dynamics

Before analyzing the properties of the utility function U, we justify its structure from first principles. In its current form:

$$U(E, C, R) = w_1 \cdot f(E) + w_2 \cdot f(C) + w_3 \cdot f(R) \quad (B.1)$$

may appear as a convenient aggregation. This section establishes that its structure is not arbitrary but follows from three minimal requirements on how performance, consistency, and epistemic humility combine to determine utility.

B.1 Additive Linear Structure from Separability and Scaling

We seek a utility function $U(E, C, R)$ over three measurable dimensions: efficacy E, confidence C, calibration R, satisfying the following axioms:

B1 (Monotonicity): U is strictly increasing in each argument.

B2 (Boundedness): U is continuous on $[0, 1]^3$.

B3 (Marginal Independence in Separability): The marginal effect of improving one dimension does not depend on the current level of the others. Formally for any E, E', C, R:

$$U(E, C, R) - U(E', C, R) = U(E, C', R) - U(E', C', R)$$

B4 (Scale Invariance): The functional form of U is identical across fields, only the weight vector (w_1, w_2, w_3) differs.

B5 (Linear Scaling Invariance): For all $\lambda \geq 0$, μ and ν that $U(E, \lambda C, \lambda R) = \lambda U(E, C, R) + \mu + \nu$

Monotonicity (B1) ensures that scaling all dimensions by the same positive factor preserves ordering. This is a weak prior: it does not specify how the dimensions combine, only that more is better. Boundedness (B2) prevents unbounded utility from dominating decisions. Scale invariance (B4) allows the same utility structure to be compared across different domains with different units.

Theorem B.1

Under axioms B1–B5, the utility function is necessarily of the form:

$$U(E, C, R) = w_1 f_1(E) + w_2 f_2(C) + w_3 f_3(R)$$

with $w_1, w_2, w_3 \geq 0$ and $f_1(E) = E$, $f_2(C) = a_2 C + b_2$, $f_3(R) = a_3 R + b_3$.

Proof

Step 1 — Additive representation from B3–B5:

B3 states that each argument is positionally independent of the others. Applying the Pexider-Aczél functional equation [1] to B3 with B4 (scale invariance) and B5 (linear scaling) yields a decomposition into three functions f_1, f_2, f_3 such that $U(E, C, R) = f_1(E) + f_2(C) + f_3(R)$.

Step 2 — Linearity from B5 alone:

Substitute the additive form into B5:

$$f_1(E) + f_2(\lambda C) + f_3(\lambda R) = \lambda f_1(E) + \lambda f_2(C) + \lambda f_3(R) + \mu + \nu$$

Let $a(\lambda) = f_2(\lambda) - \lambda f_2(1)$ and $a(\lambda) = f_3(\lambda) - \lambda f_3(1)$.

The right-hand side depends only on C and R, not on E. Therefore the left-hand side must also be constant in E:

$$f_1(E) - \lambda f_1(E) = b(\lambda) \quad \text{for all } E \in [0, 1]. \quad (1)$$

where $b : [0, \infty) \to \mathbb{R}$ is a function of λ alone. We now determine $a(\lambda)$ and $b(\lambda)$ using continuity + no affine/quadratic is assumed.

Step 2a — Determine a via Jensen's equation:

B5 gives $a(\lambda) + a(\mu) = a(\lambda\mu) + a(1) + a(\mu) + a(1)$ and $a(\lambda) + a(\mu) = a(\lambda\mu) + a(1)$.

By symmetry (swapping λ, μ):

$$a(\lambda\mu) + a(1) = a(\lambda\mu) + a(1) + a(\lambda) + a(\mu)$$

Cancelling the two equations:

$$a(\lambda) + a(\mu) = a(\lambda\mu) + a(1) \quad (2)$$

Let $g(t) = a(e^t) - a(1)$. Then (2) becomes $g(s) + g(t) = g(s + t)$. Continuity + (2) → $g(t) = ct$ for some constant c. So $a(\lambda) = c \ln \lambda + a(1)$. This derivation uses only continuity (B2) to differentiability of a, no affine/quadratic.

Step 2b — Constrain linearity of f_1, f_2, f_3 via B5:

Substituting $a(\lambda) = c \ln \lambda + a(1)$ into B5:

$$c \ln \lambda + c \ln \mu + c \ln \nu = c \ln(\lambda\mu\nu) + c \ln \mu + c \ln \nu$$

Cancelling $c \ln \mu + c \ln \nu$ from both sides: $c \ln \lambda + c \ln \nu = c \ln(\lambda\mu\nu)$, and by continuity of $\ln$: $c \ln \lambda + c \ln \mu = c \ln(\lambda\mu)$. Exponentiating: $\lambda\mu = \lambda\mu$. ✓ Holds for all c.

Step 3 — Normalization:

With $f_1(E) = E$, we get $w_1 = w_1 \cdot 1$. Under the functional form in the same across fields, so field dependence enters only through w_1, w_2, w_3. Finally, f_2 and f_3 are affine: $f_2(C) = a_2 C + b_2$, $f_3(R) = a_3 R + b_3$. Absorbing a_2 into w_2 and a_3 into w_3 and dropping constants: $U(E, C, R) = w_1 E + w_2 C + w_3 R$. ✓

Therefore:

$$U(E, C, R) = w_1 f_1(E) + w_2 f_2(C) + w_3 f_3(R) \quad \blacksquare$$

Remark on Non-Additive Interactions

Real capabilities form a hierarchy: E enables C (high efficacy makes calibrating of increasingly E depends on C and R). Ideally, the functional form would include terms like $w_{12} E \cdot C$ or $w_{23} C \cdot R$. However, B3 (marginal independence) rules out such interactions. The additive form is the simplest approximation that preserves monotonicity and boundedness. The hierarchy is not ignored, it is the unique form satisfying all five axioms jointly.

B.2 Field-Specific Weighting via Cost Sensitivity

Different domains place different importance on correctness, reliability, and exploration. We model this as a field-dependent weight vector (w_1, w_2, w_3).

Setup. Define c_E (cost of a missed success) and c_C (cost of a false claim). Let $\rho = c_E / c_C$ (cost of missed opportunity) and $\sigma = c_C / c_E$ (expected cost of being in expectation). Assume $\rho \geq 1$ (missed opportunities are at least as costly as false claims).

Design principle. We set

$$w_1 = \frac{\rho}{1 + \rho + \sigma}, \quad w_2 = \frac{1}{1 + \rho + \sigma}, \quad w_3 = \frac{\sigma}{1 + \rho + \sigma}$$

so that the gradient $\nabla U = (w_1, w_2, w_3)$ is proportional to the cost vector. This ensures that a unit improvement in the highest-cost dimension produces the largest utility gain, aligning the system's optimization with deployment cost.

Empirical validation. The weight ordering is verified against professional liability standards:

Field	ρ	σ	w_1 (E)	w_2 (C)	w_3 (R)
Medical	10.0	0.1	0.826	0.083	0.091
Legal	5.0	0.2	0.714	0.143	0.143
Publishing	2.0	0.5	0.571	0.286	0.143
Exploratory	1.0	1.0	0.333	0.333	0.333

Sub	$\mathbb{R}\times\mathbb{R}\times\mathbb{R}\times\mathbb{R}\to\mathbb{R}$	Subtraction	$(a,b,c,d)\mapsto a-b$
Inf(Lower Bound)	$\mathbb{R}\times\mathbb{R}\to\mathbb{R}$	Infimum	$(a,b)\mapsto\inf\{a,b\}$
Sup(Upper Bound)	$\mathbb{R}\times\mathbb{R}\to\mathbb{R}$	Supremum	$(a,b)\mapsto\sup\{a,b\}$
Cos	$\mathbb{R}\to\mathbb{R}$	Cosine	$x\mapsto\cos x$
Sin	$\mathbb{R}\to\mathbb{R}$	Sine	$x\mapsto\sin x$

The weight ordering $\succ_{\text{lexicographic}}$ is consistent with medical malpractice lawsuits. ICAO Annex 13 accident reports and IAEA SSG2 software safety classifications, all of which place safety consequence requirements higher priority than reliability.

Status: This is a decision-theoretic design principle grounded in cost proportionality, not a mere convention. Under the lexicographic ordering, no reliability improvement can compensate for a safety harm, ensuring that a Force Majeure exceedance event minimizes cost in a specified loss model.

9.1 Efficiency as a Governing Performance Metric

We define efficiency as a function of the realized performance ratio:

$$\eta(\omega) = \frac{\text{Actual Performance}(\omega)}{\text{Design Baseline}}$$

using the transformation:

$$\Phi(x) = \frac{1}{1+x}$$

Properties

Property	Formula	Value
Maximum value (zero harm exceedance)	$\Phi(0)$	1
Asymptotic decay	$\lim_{x\to\infty}\Phi(x)$	0
Bounded factor	$\forall x\geq 0:\ \Phi(x)\leq 1$	
Monotonic decay	$x_1<x_2\Rightarrow\Phi(x_1)>\Phi(x_2)$	
Inverse recovery (zero-harm condition)	$\Phi^{-1}(1)=0$	

Name: Pareto Interpretation

Under the log-logistic performance model, $\Phi(x)$ equals the Harr-Wittmeg dominance probability exactly — not merely analogously.

Proposition 9.1. Let $X_{\text{perf}}\sim\text{LogLogistic}(\alpha,\beta)$ and $X_{\text{harm}}\sim\text{LogLogistic}(\alpha,\gamma)$ with the same scale $\alpha>0$, and $\gamma>\beta>0$ for the ratio of shapes. Then

$$\Pr(X_{\text{perf}}>X_{\text{harm}}) = \frac{1}{1+\left(\frac{\alpha}{\alpha}\right)^{\gamma}} = \Phi(0)$$

Proof The log-logistic distribution has CDF $F(x;\alpha,\beta)=\frac{x^{\beta}}{\alpha^{\beta}+x^{\beta}}$ for $x\geq 0$.

$$\Pr(X_{\text{perf}}>X_{\text{harm}}) = \int_0^\infty f_{\text{harm}}(x)\,\Pr(X_{\text{perf}}>x)\,dx$$

Under the log-logistic model with $\alpha=\alpha_{\text{perf}}=\alpha_{\text{harm}}$ this integral yields the closed-form expression (under the log-logistic assumption):

$$\Pr(X_{\text{perf}}>X_{\text{harm}}) = \frac{1}{1+\left(\frac{\alpha}{\alpha}\right)^{\gamma}} = \frac{1}{1+1^{\gamma}} = \Phi(0)$$

Note: The result uses the specific structure of the log-logistic CDF. The difference of log-logistic random variables does not in general follow a logistic distribution; the characteristic arrow reversal in evaluating the integral under this model, not from a distribution-free argument, is specific to log-logistic.

Scope of the claim. The equality $\Phi(x)=\Pr(X_{\text{perf}}>X_{\text{harm}})$ holds under the log-logistic model with equal scale parameters (distributional assumption) (e.g. normal), the dominance probability takes a different form. The log-logistic assumption is standard for ratio comparisons in survival analysis and reliability engineering, where it models multiplicative degradation processes. Under $\alpha=\alpha_{\text{perf}}=\alpha_{\text{harm}}$ the ratio γ/β controls the separation; the shape of distribution is a modeling assumption, not a mathematical necessity.

Comparison to Classic Formulation

Classic reliability theory uses $\Phi(x)$ as a discriminative derivative $\omega=\frac{1}{1+x}$ given only marginal profiles for the superior/inferior impairment, and lacks a probabilistic interpretation. The signed form $\eta=\frac{1}{1+x}$ is valuable of three issues and is the natural functional form for a dominance probability under a location-scale family of performance distributions.

9.2 Confidence as a Scalar Optimal Filtered Estimate

Confidence is updated as the exponential moving average:

$$C_t = (1-\lambda)C_{t-1} + \lambda\,\eta_t$$

where $\eta_t\in[0,1]$ is the observed test pass rate at time t.

Start Space Model

Model drift (known collapse) & Elite agents's true underlying competence) as a random walk observed through noisy pass rate:

$$\theta_{t+1} = \theta_t + \epsilon_t,\quad \epsilon_t\sim\mathcal{N}(0,\sigma_\epsilon^2)\quad\text{(process noise)}$$

$$\eta_t = \theta_t + \epsilon_t,\quad \epsilon_t\sim\mathcal{N}(0,\sigma_\epsilon^2)\quad\text{(observation noise)}$$

The random walk noise captures the assumption that true competence changes gradually — through collaboration and learning — rather than jumping discontinuously.

Theorem 9.2 (Kalman EMA Equivalence).

At steady state, the Kalman filter for the above system reduces exactly to the EMA update $C_t = (1-\lambda)C_{t-1} + \lambda\eta_t$ with optimal gain:

$$\lambda^* = \frac{-\sigma_\epsilon^2 + \sqrt{\sigma_\epsilon^4 + 4\sigma_\epsilon^2\sigma_\epsilon^2}}{2\sigma_\epsilon^2}$$

Interpretation of λ

The noise ratio $\kappa=\sigma_\epsilon/\sigma_\epsilon$ determines process noise is 5% of observation noise. true competence changes slowly relative to the variability of individual test outcomes. Its the correct regime for incremental submission pass rate interactions — a single test pass or fail is noisy, while genuine competence changes across many submissions. Setting $\lambda=0.05$ is not a tuning choice; it is the Kalman optimal gain for an agent whose true competence evolves at 5% the rate of observational variability.

Instability

$$\omega = C_t^{\alpha}\quad\text{Optimal}\to\text{Regime}$$

$[0,0.5)$	$\omega>1$	Stable, high-confidence regime — accelerate
$[0.5,1.0)$	$0<\omega<1$	Intermediate regime. Observation
0.5	$\omega=1$	Marginal stability — critical threshold
$(1.0,\infty)$	$\omega<0$	Post-collapse regime — intervention

Its high-value fields where competence changes very slowly (surgery, aviation), smaller ω values are appropriate. Setting field-specific optimal gain from observed long-run variance is left as future work.

Remark on reasoning direction. The derivation above runs in reverse: the value $\eta=\frac{1}{1+x}$ was chosen because it satisfies the dominance probability interpretation, and the Kalman filter then recovered exponential smoothing as the unique optimal estimator for this system. The reverse is true: this choice is principled, optimal. This is a justification of a priori heuristic rather than a post-derivation. An alternative framing: if we initialize the process in observation space with a prior $\eta\sim\mathcal{U}[0,1]$, λ can be interpreted as changes as roughly 5% the rate of single trial variability when it is stable, and faster when it is unstable. Whether this prior is correct for a given domain is an empirical question. The asymmetry takes confirm the Kalman direction is consistent: $\lambda=0.05$ returns $C_t=1.00$ basically. The validity of the Recent derivation is model-dependent. Under a different noise structure, the optimal filter would differ. Under heavy-tailed shocks non-stationarity, e.g. Rosso, robust observation noise, or non-stationary competence dynamics.

9.3 Confidence as a Cost-Sensitive Exponential Decay

We define

$$\Phi(x) = \frac{1}{1+x},\quad C(\eta,\alpha) = \log_2(1+\eta\alpha)$$

where C_{raw} is the remaining confidence gap, $\alpha\geq 0$ is the severity of domain d and η_{eff} is the effective learning rate (recovery (cases or scale proxies).

Structural Analogy to KTD

The UCB algorithm (Auer et al. 2002) selects arms by:

$$a_t = \arg\max_a\left(\hat{\mu}_a + \sqrt{\frac{\log t}{N_a}}\right)$$

The curiosity term maps onto this structure as follows:

UCB component	Curiosity component	Interpretation
$\hat{\mu}_a$ (mean reward)	η_{eff} (recovery)	Learned reward/recovery
$\sqrt{\log t/N_a}$ (exploration bonus)	$C_{\text{raw}}=\frac{1}{1+\eta\alpha}$	Remaining uncertainty in domain d
$\sqrt{\log t/N_a}$	$\Phi(x)=\frac{1}{1+x}$	Monotonic decay function

Both increases are concave and increasing in the "time since last exploration," creating persistence pressure that prevents premature convergence. The curiosity term is a domain-specific instance of a bounded-from-0 UCB bonus $\sqrt{\log t/N_a}$ with C_{raw} as the exploration bonus. Both serve the sublinear growth family that prevents any single domain from dominating selection indefinitely.

What this achieves. The curiosity term in UCB implies σ drives the structural properties (concavity) driven, concave is familiarity, bounded by exploitation) that make UCB effective. We inherit the theoretical guarantees (sublinear regret, no premature convergence) from decades of bandit research rather rule-specific tuning which will be brittle.

Proposition 9.4 – The Cap Enforces Exploration Dominance

Proposition. The constraint $\eta_{\text{eff}}^{\alpha}\leq\alpha_{\text{cap}}$ implies that curiosity contribution at most 50% of total utility at all times:

$$\eta_{\text{eff}}^{\alpha}\leq\alpha_{\text{cap}}\Rightarrow\frac{C_{\text{raw}}}{1+C_{\text{raw}}}\leq\frac{1}{2}$$

Proof Let $X=\eta_{\text{eff}}^{\alpha}$. Then $C_{\text{raw}}=\frac{1}{1+X}$. The ratio simplies to $\frac{C}{1+C}$. Total utility is $U=1+\eta_{\text{eff}}$. Therefore

$$\frac{C_{\text{raw}}}{U} = \frac{1}{2(1+X)}$$

Equality holds only when $\eta_{\text{eff}}=1$. If η_{eff} drops sufficiently it is at the maximum and exploitation contribution equals 50% of all other terms $\eta_{\text{eff}}=1.0$.

Why 50%?

The 50% threshold is the highest constant upper bound derivable from the single constraint $\eta_{\text{eff}}^{\alpha}\leq\alpha_{\text{cap}}$ without additional assumptions. Higher caps (e.g. 75%) would be unreasonably restrictive (exploration being early/mixing phase of α and α drawn from log-$\mathcal{N}(0,\sigma^2)$ and exploitation is dominate even when the agent has high confidence and efficiency η_{eff} is high). Lower caps (e.g. 25%) would allow exploitation to dominate too quickly, risking premature convergence. The 50% cap is the structural compromise aligned with the requirement that exploration remains dominant.

What remains open. Whether the cap should be fixed or adaptive (where optimal signal guarantees under the multi-armed bandit formulation — including formal minimax bounds — is an open question. The analogy to UCB provides intuition and motivation, and the theoretical properties (monotonicity, bounded-from-0, finite total regret) analogous to classical work.

9.4 Summary

The pass-rate transformed $\eta=\Phi(\cdot)$ and C_t is profiled as follows:

Component	Justification	Status
Monotonic decay	Harder tasks / lower recovery yield lower curiosity	Theorem (proven)
Bounded by exploitation	Curiosity functional approaches 0 as exploitation saturates	Theorem (proven)
Finite regret	Inherit from underlying UCB structure	Inherited
Sublinear growth	Inherit from UCB structure	Inherited
Without $\Phi(\cdot)$ (raw C_{raw})	Curiosity dominates indefinitely	Provable counterexample
Cap at 50%	Exploration dominance	Proposition
Optimality	Not-triangular, equilibrium-dependence general	Open/not in general

The formulation is not claimed to be the unique curiosity design, but is the **minimal interpretable and theoretically grounded** decay component with the four properties. Each component rests on an identified theoretical foundation, and the scope of each claim is stated explicitly.

9.5 — Theorem 9.5 – Convergence of Confidence Under Repeated Collaboration

Setup

Recall the confidence update rule with contraction parameter

$$C_t = (1-\lambda)C_t + \lambda\,\eta(\tau)$$

where $\tau\sim\mathcal{U}(\{1,\ldots,t\})$ the EMA learning rate, $\eta\in[0,1]$ the observed test pass rate at time τ, $\lambda\in(0,1)$ is the contraction penalty magnitude (zero when no contradiction occurs), $\eta(\tau)$ is the field penalty multiplier, τ denotes the across field.

Define the **effective signal** $\eta_t=\eta(\tau)-\lambda\eta(\tau)$ When no contradiction occurs $\lambda=0$ and $\eta_t=\eta_t$. When a contradiction occurs (indicator $\mathbb{1}$ is relevant), it is reduced by a factor $(1-\lambda)$.

$$\mathbb{E}[\eta_t] = (1-\lambda)\mu^*+\lambda\mu_{\text{contr}}^*$$

Kal Similarity (or cosine similarity)

$$\text{sim}(\tau_1,\tau_2) = \cos(\theta) = \frac{\langle\tau_1,\tau_2\rangle}{\|\tau_1\|\,\|\tau_2\|}$$

where $\tau_1=\mathbb{1}[\text{Kal}]$ is the EMA learning rate, $\tau_2=\mathbb{1}$ is the indicator that pass rate at time t τ is the contradiction penalty magnitude (zero when no contradiction occurs), $\eta(\tau)$ is the field penalty multiplier, τ denotes the across field.

Proof. Induction. Base case: $t=0$, $C_0=0$, trivially satisfied. Inductive step: assume the formula holds for t. Then:

$$C_{t+1} = (1-\lambda)C_t+\lambda\eta_t$$

$$= (1-\lambda)\left((1-\lambda)\mu_t+\lambda\mu_{\text{contr},t}\right)+\lambda\mu_t$$

$$= (1-\lambda)^2\mu_t+(1-\lambda)\lambda\mu_{\text{contr},t}+\lambda\mu_t$$

$$= (1-\lambda)^2\mu_t+\lambda\mu_t-\lambda^2\mu_t+\lambda\mu_{\text{contr},t}$$

$$= \mu_t+\lambda\mu_{\text{contr},t}-\lambda^2\mu_t$$

Q.E.D.

9.6 — Theorem 9.6 – Convergence Under Contradiction

Theorem. μ_t has a stationary expansion with constant expectation $\mu=\mathbb{E}_\tau$ and variance $\sigma_\infty=\mathbb{E}[\mu_t-\mu]$ μ_t is the penalty regime is over contradiction μ_{contr} and μ_t is the penalty regime is over contradiction μ_{contr} μ_t is the expectation of μ_t μ_t is the expectation of μ_t μ_t is the expectation of $\mu_t

Part (iv) — Persistent-drift stability.

Since $s^* \in B$, the non-expansiveness of Π_B with respect to any point in B gives:

$$\begin{aligned}\|s_{t+1} - s^*\| &= \|\Pi_B[(1-\beta)s_t + \beta s^* + \Delta_t] - \Pi_B[s^*]\| \\ &\le \|(1-\beta)s_t + \beta s^* + \Delta_t - s^*\| \\ &= \|(1-\beta)(s_t - s^*) + \Delta_t\| \\ &\le (1-\beta)\|s_t - s^*\| + \|\Delta_t\| \\ &\le (1-\beta)\|s_t - s^*\| + \Delta_{\max}\sqrt{n}\end{aligned}$$

Let $d_t = \|s_t - s^*\|$. The recurrence $d_{t+1} \le (1-\beta)d_t + \Delta_{\max}\sqrt{n}$ has fixed point $d^* = \Delta_{\max}\sqrt{n}/\beta$. By the theory of contractive linear recurrences:

- If $d_t > d^*$: $d_{t+1} < d_t$ (distance decreasing toward d^*)
- If $d_t \le d^*$: $d_{t+1} \le d^*$ (distance stays within neighborhood)

So $\limsup_{t\to\infty} d_t \le d^* = \Delta_{\max}\sqrt{n}/\beta$.

Since $d_t \le \operatorname{diam}(B)$ always by Part (i), the binding constraint is:

$$\mathrm{r}^* = \min\left(\frac{\Delta_{\max}\sqrt{n}}{\beta},\ \operatorname{diam}(B)\right) \qquad \blacksquare$$

Why the Field Bounds Are the Binding Constraint

For the system parameters:

$$\frac{\Delta_{\max}\sqrt{n}}{\beta} = \frac{0.05 \times \sqrt{6}}{0.01} = \frac{0.1225}{0.01} = 12.25$$

$$\operatorname{diam}(B) = \|\, s_{\max} - s_{\min} \,\| \le \sqrt{n} \approx 2.45 \quad (\text{since each trait is in } [0,1])$$

Since $12.25 \gg 2.45$, the mean reversion alone is insufficient to confine the dynamics to a small neighborhood of s^*. **The projection Π_B is the primary stability mechanism** — it enforces invariance, and the field bounds define how far from s^* the personality can stray. The mean reversion serves as a regularizer: without it, the system would drift to and remain at the boundary of B; with it, there is a gentle restoring force toward s^* when drift is absent.

This clarifies the design role of each stability layer:

Layer	Mechanism	Guarantee
Field bounds $[s_{\min}, s_{\max}]$	Projection Π_B	Hard invariance — $s_t \in B$ always
Drift rate cap $\Delta_{\max}$	Truncation of raw drift	Bounded single-step displacement
Mean reversion β	Soft pull toward s^*	Convergence to s^* when drift is absent; regularization otherwise

---

Corollary — No Oscillation

Corollary. *The drift rate cap $\Delta_{\max}$ prevents oscillation: after any evolution step, the personality cannot cross s^* in a single step.*

Proof. A step crosses s^* in dimension i if $s^i_{t+1} - s^*_i$ and $s^i_t - s^*_i$ have opposite signs. The signed displacement in dimension i is:

$$s^i_{t+1} - s^*_i = \Pi_{[s^i_{\min}, s^i_{\max}]}\left[(1-\beta)(s^i_t - s^*_i) + \Delta^i_t\right]$$

For oscillation, we need $|\Delta^i_t| > (1-\beta)|s^i_t - s^*_i|$, i.e., the drift must exceed the current displacement. Since $\Delta^i_t \le \Delta_{\max} = 0.05$ and $s^i_t - s^*_i$ can be as large as $s^i_{\max} - s^*_i$ (which is at least 0.05 by the bound structure), oscillation requires the personality to be within $\Delta_{\max}/(1-\beta) \approx 0.0505$ of s^* in that dimension. This is a vanishingly small region, confirming that oscillation can only occur when the personality is already very near the neutral point. ■

Remark on Parameter Calibration

The analysis reveals a structural tension: mean reversion $\beta = 0.01$ is weaker than the maximum drift $\Delta_{\max} = 0.05$ per cycle. This is intentional — personality is meant to evolve meaningfully, not snap back to s^* every few cycles. The design trades off:

- **Large** β: fast convergence to s^*, but personality becomes unresponsive to experience
- **Small** β: personality evolves with experience, but reverts slowly between periods of drift

The value $\beta = 0.01$ produces a half-life of approximately 34 evolution cycles under zero drift (≈ 102 interactions), which is long enough for the personality to reflect accumulated experience over hundreds of interactions before the neutral pull becomes dominant. A field-specific $\beta(f)$ — with higher reversion rates for high-stakes fields where personality stability is more important — is a natural extension for future work.

References

Note on scope: This bibliography has been expanded to better match the breadth of the framework. It is still an evolving list, but now covers the main theoretical, systems, deployment, privacy, hardware, autonomy, and mechanism-design strands explicitly discussed in the paper.

- Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3), 235–256.
- Blackorby, C., & Donaldson, D. (1982). Ratio-scale and translation-scale full interpersonal comparability without domain restrictions. *International Economic Review*, 23(2), 249–268.
- Cavoukian, A. (2009). *Privacy by Design: The 7 Foundational Principles.*
- Clarke, E. H. (1971). Multipart pricing of public goods. *Public Choice*, 11(1), 17–33.
- Debreu, G. (1960). Topological methods in cardinal utility theory. In K. J. Arrow, S. Karlin, & P. Suppes (Eds.), *Mathematical Methods in the Social Sciences, 1959*. Stanford University Press.
- Fedus, W., Zoph, B., & Shazeer, N. (2021). Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120), 1–39.
- GDPR. (2016). Regulation (EU) 2016/679 of the European Parliament and of the Council.
- Groves, T. (1973). Incentives in teams. *Econometrica*, 41(4), 617–631.
- Harsanyi, J. C., & Selten, R. (1972). A generalized Nash solution for two-person bargaining games with incomplete information. *Management Science*, 18(5-part-2), 80–106.
- Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation.* Addison-Wesley.
- Hurwicz, L. (1972). On informationally decentralized systems. In C. B. McGuire & R. Radner (Eds.), *Decision and Organization*. North-Holland.
- ISO. (2018). *ISO 26262 Road Vehicles — Functional Safety.*
- Johari, R., & Tsitsiklis, J. N. (2004). Efficiency loss in a network resource allocation game. *Mathematics of Operations Research*, 29(3), 407–435.
- Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1), 35–45.
- Kelly, F. P. (1997). Charging and rate control for elastic traffic. *European Transactions on Telecommunications*, 8(1), 33–37.
- Kindervag, J. (2010). *No More Chewy Centers: Introducing the Zero Trust Model of Information Security.* Forrester Research.
- Koutsoupias, E., & Papadimitriou, C. H. (1999). Worst-case equilibria. In *STACS*.
- Lamport, L. (1998). The part-time parliament. *ACM Transactions on Computer Systems*, 16(2), 133–169.
- Li, X., et al. (2022). Branch-Train-Merge: Embarrassingly parallel training of expert language models. *arXiv preprint.*
- Mann, H. B., & Whitney, D. R. (1947). On a test of whether one of two random variables is stochastically larger than the other. *Annals of Mathematical Statistics*, 18(1), 50–60.
- Nash, J. F. (1950). The bargaining problem. *Econometrica*, 18(2), 155–162.
- NHTSA. (2020). *Automated Vehicles 4.0: Ensuring American Leadership in Automated Vehicle Technologies.*
- NIST. (2020). *Special Publication 800-207: Zero Trust Architecture.*
- NVIDIA. (2024–2025). NVLink / NVSwitch and GPU interconnect technical documentation and performance materials.
- Ongaro, D., & Ousterhout, J. (2014). In search of an understandable consensus algorithm. In *USENIX ATC.*
- OpenAI. (2024). Direct Preference Optimization and related preference-optimization methods. Product and technical publications.
- Roughgarden, T., & Tardos, É. (2002). How bad is selfish routing? *Journal of the ACM*, 49(2), 236–259.
- Saltzer, J. H. (1974). Protection and the control of information sharing in multics. *Communications of the ACM*, 17(7), 388–402.
- Shazeer, N., Mirhoseini, A., Maziarz, K., et al. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *ICLR.*
- Vickrey, W. (1961). Counterspeculation, auctions, and competitive sealed tenders. *Journal of Finance*, 16(1), 8–37.
- Wald, A. (1945). Sequential tests of statistical hypotheses. *Annals of Mathematical Statistics*, 16(2), 117–186.
- Wang, X., et al. (2022). Self-consistency improves chain of thought reasoning in language models. *arXiv preprint.*
- European Union. (2024). *EU AI Act.*
- Mixtral and related mixture-of-experts production-model technical reports (2023–2024).